

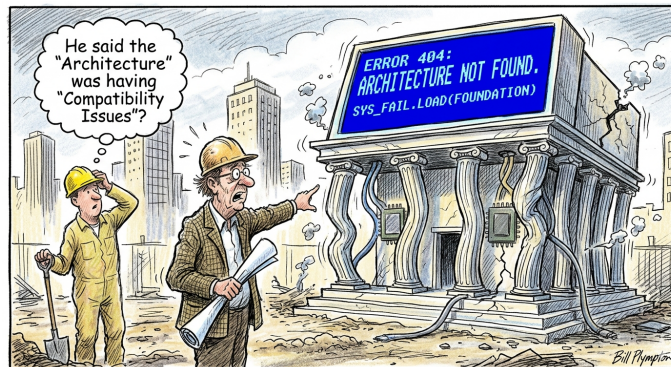
ARQUITECTURA Y ESPECIFICACIONES

Sistema RTM32

Manual del Usuario y Especificación de Arquitectura

Diseñado para alta velocidad de ejecución y modularidad.

Alejandro Rodriguez Costello





UNIVERSIDAD NACIONAL DE ROSARIO

Instituto Politécnico Superior

Cátedra de Arquitectura de Computadoras

Documento: Manual de Usuario y Datos Técnicos
Versión: 0.1.0-RC1 (*first release condidate*)
Fecha: 27 de mayo de 2026
Contacto: `acrod@ips.edu.ar`

Licencia y Derechos de Autor

© 2026 Alejandro Rodriguez Costello.

Este documento se publica bajo la licencia **Creative Commons**

Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0).

Para ver una copia de esta licencia, visite <http://creativecommons.org/licenses/by-nc-sa/4.0/>.

Parte I

Arquitectura del Sistema RTM32

1. Introducción

1.1. Generalidades

La aplicación **RTM32** es un emulador de la arquitectura RISC de 32 bits **TM32** optimizada nativamente mediante técnicas de enlazado directo estático (Direct Threaded Code) e inlining agresivo de memoria para priorizar la velocidad de ejecución.

Este manual proporciona el modelo de programación a nivel de sistema y de uso para aplicación. El propósito principal de esta documentación es guiar a los desarrolladores en la escritura de software a nivel ensamblador (*assembly*), la construcción de boot ROMs y la posibilidad de portar un compilador de un lenguaje de alto nivel (HLL) como Oberon. Este texto establece las convenciones de software, manejo de la memoria y los requisitos para la utilización de registros.

El alcance de este documento se limita estrictamente a la interfaz arquitectónica visible por software y no profundiza en la mecánica interna del emulador. En su lugar, cubre los límites estructurales del conjunto de instrucciones, los 32 registros de propósito general y 7 de propósito especial, la jerarquía de memoria y los mecanismos formales de manejo de excepciones e interrupciones por hardware.

El emulador equilibra claridad educativa con una ejecución veloz diseñada para tener rendimiento en la ejecución de sistemas operativos, aplicaciones de consola y emulación de videojuegos, en lugar de una simulación precisa de ciclo (*cycle-accurate physical simulation*). Componentes complejos de hardware como la unidad de manejo de memoria (MMU) paginada se omiten en favor de regiones de memoria planas alineadas. Los estados internos se condensan en representaciones directas mediante flags, resultando en un modelo de ejecución simplificado y fácil de entender para programadores de compiladores y estudiantes de computación por igual.

Esta arquitectura del emulador está diseñada para ejecutar un stack de software especializado. El despliegue inicial se centra en el diseño de boot ROMs monolíticas de bajo nivel para configurar el sistema (*first stage*) y posibilitar la carga y ejecución de un bloque de código en almacenamiento externo (*boot loader*) para que se encargue de cargar el resto del sistema (*second stage*). Uno de los objetivos a largo plazo es portar completamente el entorno del sistema operativo **Oberon** y su lenguaje compilador orientado a objetos, adaptando el concepto de estación de trabajo a una estructura de aplicación de consola liviana y de alto rendimiento.

1.2. Arquitectura del emulador

La arquitectura **RTM32** se compone de tres software de aplicación llamados: **RTM32**, el emulador de la máquina, **RTM32AS** el compilador cruzado (*cross compiler*) del lenguaje ensamblador de la arquitectura **TM32** y una utilidad llamada **RTM32FMT** para administrar los dispositivos de masa. La máquina está compuesta por una CPU, un manejador de interrupciones, un bus de 32 bits unificado, como se observa en la figura 1.1, RAM, ROM y distintos periféricos optativos.

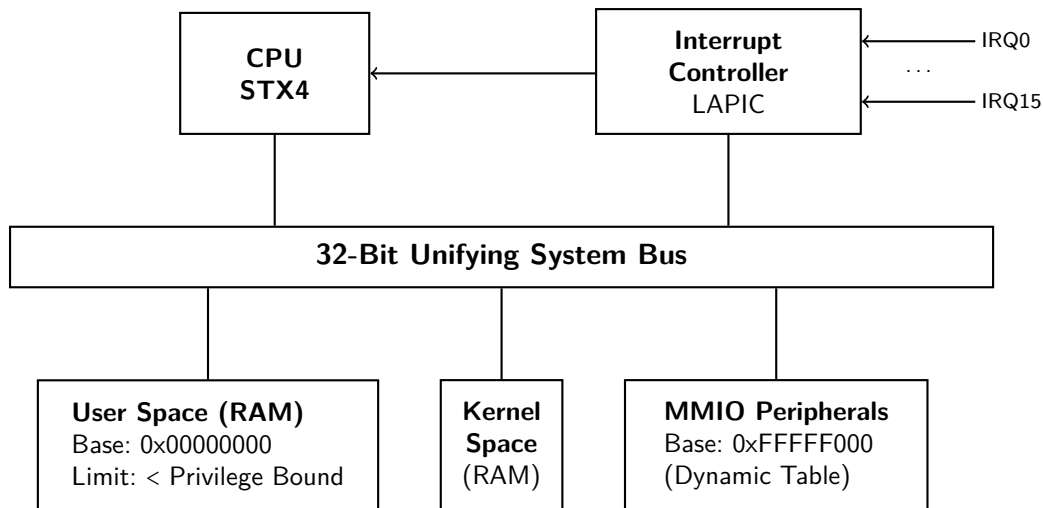


Figura 1.1: 32-Bit Unifying System Bus Architecture.

Su configuración se realiza a través de un archivo descriptivo y switches opcionales que se pueden desplegar en la línea de comando como se describe en el Capítulo 2.

Para avanzar en el desarrollo de aplicaciones se dispone del compilador **RTM32AS** descrito en el Capítulo 9.1, que genera código objeto ejecutable en el emulador. Dicho objeto se puede cargar desde la inicialización o desde el modo depurador (*debug*) descrito en el Capítulo 10.

La utilidad **RTM32FMT** nos permite crear un archivo en el host que represente una unidad de almacenamiento como un disco, formatearlo y editar su contenido en forma offline para anexarlo al controlador de dispositivo correspondiente durante la inicialización (*disc attach*).

El modelo de programación cambia entre dos posibles estados con diferentes privilegios (*priviledge states*):

- Modo **Kernel**: o privilegiado (*priviledge*), con acceso completo al ISA y a la memoria mapeada disponible.
- Modo **User**: restringido a ciertas instrucciones y regiones de la memoria mediante excepciones (*traps*).

La representación de datos en todos los buses, espacios de memoria y regiones de acceso a los dispositivos es estrictamente *Little-Endian*.

1.3. Procesador (STX4)

El procesador central del sistema RTM32 es el **STX4**, un procesador RISC de propósito general de 32 bits que implementa la arquitectura de conjunto de instrucciones (ISA) RTM32. El procesador ejecuta un único flujo de instrucciones sobre un espacio de direcciones unificado de 32 bits, compartido por el código ejecutable, los datos y los dispositivos de entrada/salida mapeados en memoria (MMIO).

El STX4 es un procesador de un solo núcleo (*single-core*) con ejecución en orden (*in-order*) y emisión simple (*single-issue*). Las instrucciones se obtienen, decodifican y ejecutan secuencialmente, proporcionando un comportamiento arquitectónico determinista adecuado para programación de sistemas, desarrollo de compiladores y fines educativos.

La arquitectura sigue el modelo clásico *load/store*, en el cual todas las operaciones aritméticas y lógicas se realizan exclusivamente sobre registros, mientras que el acceso a memoria se efectúa únicamente mediante instrucciones explícitas de carga y almacenamiento.

El estado arquitectónico visible por software está compuesto por treinta y dos registros de propósito general, cinco registros de propósito especial, el estado de privilegio actual y la información de control necesaria para soportar excepciones, interrupciones y transferencias de control. Estos elementos se describen en detalle en los capítulos siguientes.

La arquitectura no define cachés de instrucciones ni de datos visibles para el software. Todas las operaciones de memoria se realizan sobre un único espacio de direcciones unificado, independientemente de la implementación interna del emulador.

Las instrucciones son de ancho fijo (32 bits), estrictamente alineadas a límites de 4 bytes, y se cargan (*fetch*) secuencialmente mediante el **Program Counter** (pc).

1.4. Memoria principal

El procesador STX4 opera sobre un espacio de direcciones lineal de 32 bits, proporcionando un total de 2^{32} bytes (4 GiB) de espacio direccionable. Este espacio constituye una vista arquitectónica uniforme utilizada para acceder tanto a memoria como a dispositivos de entrada/salida mapeados en memoria (MMIO).

La cantidad de memoria física instalada no está determinada por la arquitectura. Tanto la capacidad de la memoria RAM como la de la memoria ROM son parámetros configurables del emulador y pueden variar entre distintas configuraciones del sistema sin modificar el ISA ni el modelo de programación.

La memoria RAM se divide lógicamente en dos regiones independientes: una región destinada al espacio de usuario (*User Space*) y otra destinada al espacio privilegiado (*Kernel Space*). Ambas regiones poseen rangos de direcciones distintos dentro del espacio de direcciones de 32 bits y pueden configurarse de manera independiente durante la inicialización del sistema.

La memoria ROM contiene el código de inicialización del sistema y la tabla de vectores utilizada durante el proceso de arranque, las excepciones y las interrupciones. Su contenido se carga al iniciar el emulador y permanece inmutable durante toda la ejecución del sistema, ya que la arquitectura no define mecanismos para su reprogramación mediante software.

Las regiones del espacio de direcciones que no contienen memoria ni dispositivos registrados se consideran direcciones no mapeadas. Cualquier acceso a dichas regiones produce una excepción arquitectónica de acceso inválido, cuyo comportamiento se describe en el capítulo dedicado al manejo de excepciones.

1.4.1. Segmentación del espacio de memoria

El subsistema de memoria maneja la decodificación de direcciones dividiendo su espacio en dos grandes regiones basadas en el valor del más significativo (bit 31). A la dirección límite 0x80000000 se la conoce como límite privilegiado (*Privilege Boundary*). Este límite está cableado y no se puede cambiar. Las regiones son:

- User Space (0x00000000 a 0x7FFFFFFF): Accesible universalmente por ambos modos de ejecución: Kernel Mode y User Mode. Las direcciones dentro de este segmento mapean

directamente a la RAM física volátil del sistema disponible.

- **Kernel Space (0x80000000 a 0xFFFFFFFF):** El acceso está estrictamente limitado a la ejecución en Kernel mode. Cualquier intento de un programa dentro de este espacio de direcciones en User mode o de emitir una instrucción load o store que acceda dentro de este rango dispara inmediatamente al procesador a un trap de excepción. Esta región a su vez se subdivide en:
 - **Kernel RAM Mapping:** región de la RAM mapeada a partir del inicio del Kernel Space y que se extiende hasta el comienzo de la ROM.
 - **Boot ROM Mapping:** El **Cold Boot Vector** del sistema reside en la parte superior del kernel space en 0xF0000000.
 - **Memory-Mapped I/O (MMIO):** La comunicación con hardware depende enteramente de bloques periféricos independientes registrados dinámicamente en el bus del sistema. Los registros de control de hardware por defecto se agrupan dentro del bloque superior comenzando en 0xFFFFF000.

Por simplicidad, las reglas de alineación de datos se aplican estrictamente. No hay soporte por hardware para accesos desalineados (*unaligned access*).

- **Alineación estricta de 16 bits:** Las operaciones con halfwords (**lh**, **lhx**, **lhu**, **lhux**, **sh**) requieren que la dirección de memoria destino sea múltiplo de 2 (el bit menos significativo debe ser 0).
- **Alineación estricta de 32 bits:** Las operaciones con words (**lw**, **lwx**, **lh**) y los fetch de instrucciones mediante el **pc** requieren direcciones múltiplo de 4 (los dos bits menos significativos deben ser 00).
- **Enforcement de alineación:** Cualquier operación de lectura o escritura no alineada viola el cumplimiento arquitectónico, detiene el flujo actual de ejecución y dispara un trap de hardware **ALIGNMENT_FAULT** de forma inmediata.
- **Precisión extendida:** La ejecución aritmética opera nativamente sobre registros de 32 bits. La precisión extendida de 64 bits se asigna exclusivamente, y es modificada por, las operaciones de multiplicación (**mul**, **mulh**, **mulhu**) y división (**div**, **divu**, **rem**, **remu**) de hardware de enteros.

1.4.2. Controlador de interrupciones

El sistema RTM32 incorpora un controlador local de interrupciones denominado **LAPIC** (*Little Advanced Programmable Interrupt Controller*), una implementación de la arquitectura de controlador de interrupciones **IIC** (*Intermediate Interrupt Controller*).

El controlador local de interrupciones LAPIC es el componente responsable de arbitrar todas las solicitudes de interrupción por hardware generadas por los dispositivos del sistema y presentarlas al procesador STX4 mediante una interfaz de interrupción única.

El controlador admite hasta dieciséis líneas físicas de interrupción (**IRQ0–IRQ15**), cada una configurable de manera independiente. Para cada canal es posible definir el modo de disparo, la polaridad de la señal, el nivel de prioridad, el vector de interrupción asociado y su estado de habilitación.

Cuando múltiples solicitudes se encuentran pendientes simultáneamente, el LAPIC resuelve automáticamente cuál debe ser atendida utilizando un mecanismo de arbitraje por

prioridad implementado completamente en hardware. El controlador también mantiene el estado de las interrupciones actualmente activas, permitiendo la ejecución anidada de rutinas de servicio cuando una solicitud de mayor prioridad interrumpe a otra de menor prioridad.

El LAPIC forma parte del espacio de dispositivos mapeados en memoria (MMIO), desde donde el software puede configurar sus canales y controlar el ciclo de vida de las interrupciones mediante un conjunto reducido de registros de control. La descripción completa de la interfaz de programación y del modelo de ejecución se desarrolla en el capítulo dedicado al subsistema de interrupciones.

Las excepciones arquitectónicas generadas por la CPU, como instrucciones inválidas, accesos desalineados o violaciones de memoria, no atraviesan el controlador de interrupciones. Estas condiciones son detectadas directamente por el procesador y despachadas mediante la tabla de vectores de excepción correspondiente.

1.4.3. Notación general

- Los valores hexadecimales se designan explícitamente usando el prefijo estándar `0x` (ej., `0xF0000000`).
- Los valores binarios se prefijan con `0b` (ej., `0b01010`).
- Los registros se representan usando el token `$` acompañado de su índice físico o alias de aplicación (ej., `$0` o `$z`).

2. Configuración del Emulador

El emulador RTM32 soporta dos mecanismos de configuración complementarios:

1. Archivo de configuración (TOML)
2. Argumentos de línea de comandos

El archivo de configuración por defecto, `rtm32.toml`, es opcional. Cuando está presente, se carga automáticamente desde el directorio de trabajo del emulador. El archivo de configuración puede definir cualquier opción que también esté disponible como argumento de línea de comandos, así como ajustes adicionales del emulador y del hardware virtual que no se pueden especificar desde la línea de comandos. Estos ajustes específicos de hardware se describen en una sección dedicada de este documento.

Los argumentos de línea de comandos siempre tienen precedencia sobre los valores definidos en el archivo de configuración. Esto incluye la opción utilizada para especificar un archivo de configuración alternativo, lo que permite reemplazar el archivo `rtm32.toml` por defecto al inicio. Si no se especifica una opción de configuración a través de ninguno de los dos mecanismos, el emulador utiliza su valor por defecto predefinido. Las opciones que requieren una entrada explícita del usuario se validan durante el inicio, y los valores de configuración inválidos dan como resultado un error de inicialización.

2.1. Orden de búsqueda de la configuración

Al inicio, el emulador localiza el archivo de configuración de acuerdo con el siguiente orden de búsqueda.

Prioridad	Ubicación	Descripción
1	Archivo de configuración especificado por <code>-config <file></code>	Utiliza el archivo de configuración solicitado explícitamente.
2	<code>./rtm32.toml</code>	Utiliza el archivo de configuración por defecto ubicado en el directorio de trabajo actual.
3	Valores por defecto integrados	Se utilizan cuando no se encuentra ningún archivo de configuración.

El archivo de configuración por defecto es opcional. Su ausencia no se considera un error, y el emulador continúa la inicialización utilizando sus valores por defecto integrados.

NOTA

El soporte nativo para la Especificación de Directorio Base XDG está planificado para una futura versión. La implementación actual busca intencionalmente solo en el directorio de trabajo actual para simplificar la distribución y la ejecución en entornos educativos.

2.2. Precedencia de configuración

Cuando una opción de configuración está definida por múltiples fuentes, el emulador resuelve el valor final de acuerdo con las siguientes reglas de precedencia, ordenadas por orden de prioridad donde la primera es la más alta y la última la más baja.

1. Argumentos de línea de comandos
2. Archivo de configuración
3. Valores por defecto integrados

En consecuencia, cada opción de línea de comandos sobrescribe el valor correspondiente definido en el archivo de configuración. Esto incluye la opción `-config`, que permite seleccionar un archivo de configuración alternativo antes de que se procese cualquier otro valor de configuración. Si una opción de configuración se omite tanto de la línea de comandos como del archivo de configuración, el emulador utiliza automáticamente su valor por defecto predefinido. Los valores de configuración se validan durante la inicialización. Los valores inválidos o inconsistentes impiden que el emulador se inicie y se informan como errores de configuración.

2.3. Argumentos de línea de comandos

Los argumentos de línea de comandos proporcionan un mecanismo conveniente para sobrescribir la configuración del emulador al inicio sin modificar el archivo de configuración. Cada opción de línea de comandos está disponible tanto en una forma larga (`-config`) como abreviada (`-c`). A menos que se indique lo contrario, ambas formas son funcionalmente equivalentes y pueden utilizarse indistintamente. Los argumentos de línea de comandos siempre tienen precedencia sobre los valores definidos en el archivo de configuración. Cuando una opción se especifica varias veces, la última aparición determina el valor efectivo. La mayoría de las opciones siguen una sintaxis común diseñada para proporcionar una interfaz de usuario consistente en todo el emulador.

Sintaxis	Descripción
<code>-switch=LIST[,OPTION...]</code>	Formato general del argumento.
<code>LIST</code>	Uno o más valores predefinidos aceptados por el argumento. Dependiendo de la opción, se puede especificar un solo valor o múltiples valores separados por comas.
<code>OPTION</code>	Parámetro opcional expresado como un par <code>key=value</code> que modifica el comportamiento de los valores seleccionados. Los múltiples parámetros se separan por comas.

Ejemplos:

```
--log=console
--log=console,file
--log=console,file,level=warning,color=false
```

Cuando un argumento soporta múltiples valores, estos se especifican como una lista separada por comas. El orden de los valores no es significativo. Los parámetros opcionales siempre siguen a la lista de valores y se expresan como pares `key=value`. Las siguientes secciones describen cada argumento de línea de comandos, sus valores aceptados, parámetros opcionales, comportamiento por defecto y su forma abreviada correspondiente.

2.3.1. `-debug, -d`

Habilita el debugger del emulador y selecciona la interfaz de comunicación utilizada para interactuar con la sesión de depuración. El debugger está deshabilitado por defecto y debe habilitarse explícitamente al inicio.

Gramática

```
--debug=MODE[, OPTION...]
```

Elemento	Descripción
MODE	Selecciona la interfaz del debugger.
OPTION	Parámetros de configuración opcionales para la interfaz seleccionada.

Modos

Modo	Descripción
console	Inicia el debugger interactivo utilizando los flujos estándar de entrada y salida.
telnet	Inicia el debugger como un servidor Telnet y espera la conexión de un cliente remoto.

Opciones

Las siguientes opciones se aplican al modo `telnet`.

Opción	Descripción	Por defecto
port=<n>	Puerto TCP utilizado por el servidor Telnet.	4444

Comportamiento por defecto

El debugger está deshabilitado a menos que se especifique explícitamente la opción `-debug`. El emulador requiere una fuente de ejecución al inicio. Esta puede proporcionarse habilitando el debugger, cargando una imagen de ROM o cargando una imagen de RAM. Al cargar una imagen, se utilizan las direcciones de carga y ejecución embebidas a menos que se sobrescriban explícitamente mediante las opciones de línea de comandos correspondientes. Si no se especifica ninguna fuente de ejecución, el emulador finaliza con un error de configuración.

Ejemplos

```
--debug=console
```

Inicia el debugger interactivo utilizando la terminal.

```
--debug=telnet
```

Inicia el debugger de Telnet en el puerto por defecto (4444).

```
--debug=telnet,port=4444
```

Inicia el debugger de Telnet utilizando el puerto TCP especificado.

2.3.2. -load, -l

Carga una imagen binaria en la memoria del emulador antes de la ejecución. Por defecto, la imagen se carga en la dirección especificada por su formato de archivo. Se puede proporcionar una dirección opcional para sobrescribir la dirección de carga embebida y reubicar la imagen en cualquier dirección válida y correctamente alineada dentro del espacio de direcciones del emulador.

Gramática

```
--load=FILE[,OPTION...]
```

Elemento	Descripción
FILE	Ruta a la imagen binaria que se va a cargar.
OPTION	Parámetros opcionales que modifican el comportamiento de carga.

Opciones

Opción	Descripción	Por defecto
address=<addr>	Sobrescribe la dirección de carga de la imagen.	Dirección embebida en la imagen

Comportamiento por defecto

La imagen se carga antes de que el emulador comience la ejecución. Cuando no se especifica ninguna dirección de carga, se utiliza la dirección embebida en la imagen. Si se proporciona una sobrescritura, la imagen se reubica en la dirección especificada. La dirección específica debe hacer referencia a una ubicación válida y correctamente alineada dentro del espacio de direcciones del emulador. Las direcciones inválidas se informan como errores de configuración.

Ejemplos

```
--load=test.bin
```

Carga la imagen utilizando la dirección embebida en el archivo.

```
--load=test.bin,address=0x2000
```

Carga la imagen en la dirección 0x2000, sobrescribiendo la dirección embebida en el archivo.

2.3.3. -log, -g

Configura el subsistema de logs del emulador. Se pueden habilitar uno o más destinos de log simultáneamente. Todos los mensajes de log generados se envían a los destinos seleccionados utilizando el mismo nivel de log y opciones de formato.

Gramática

```
--log=DESTINATION[,OPTION...]
```

Elemento	Descripción
DESTINATION	Uno o más destinos de log. Los múltiples valores se separan por comas.
OPTION	Parámetros opcionales que controlan el comportamiento de los logs.

Destinos

Destino	Descripción
display	Escribe los mensajes de log en la salida estándar.
file	Escribe los mensajes de log en un archivo de log.

Opciones

Opción	Descripción	Por defecto
file=<path>	Ruta del archivo de log de salida. Requerido cuando se selecciona el destino file .	—
level=debug info warning error	Nivel mínimo de severidad a registrar.	info
color=true false	Habilita o deshabilita las secuencias de color ANSI para la salida de la terminal.	true

Los destinos seleccionados comparten el mismo nivel de log y opciones de formato.

Comportamiento por defecto

El registro de logs está deshabilitado a menos que se especifique explícitamente la opción **-log**. Cuando se seleccionan múltiples destinos, cada mensaje de log se envía a cada uno de los destinos seleccionados.

Ejemplos

```
--log=file,file=emu.log,level=debug
```

Escribe todos los mensajes de log con severidad **debug** o superior en el archivo **emu.log**.

```
--log=display,level=warning,color=false
```

Muestra los mensajes de log con severidad **warning** o superior en la terminal sin colores ANSI.

```
--log=display,file,emu.log,level=info
```

Escribe el mismo flujo de log simultáneamente en la terminal y en **emu.log**.

2.3.4. **-exec, -e**

Establece la dirección de ejecución inicial de la CPU virtual. Por defecto, la ejecución comienza en el punto de entrada definido por la imagen cargada. Se puede proporcionar una dirección de ejecución explícita para sobrescribir el punto de entrada embebido y comenzar la ejecución desde cualquier dirección válida y correctamente alineada dentro del espacio de direcciones del emulador.

Gramática

```
--exec=ADDRESS[,OPTION...]
```

Elemento	Descripción
ADDRESS	Dirección de ejecución inicial.
OPTION	Parámetros opcionales que controlan el comportamiento de la ejecución.

Opciones

Opción	Descripción	Por defecto
steps=<n>	Cantidad máxima de instrucciones a ejecutar antes de devolver el control.	Ilimitado

Comportamiento por defecto

Cuando no se especifica la opción **-exec**, el emulador comienza la ejecución en el punto de entrada definido por la imagen cargada. Si se identifica una dirección de ejecución, esta sobrescribe el punto de entrada de la imagen. La dirección específica debe hacer referencia a una ubicación válida y correctamente alineada dentro del espacio de direcciones del emulador. Las direcciones inválidas se informan como errores de configuración.

Ejemplos

```
--exec=0x1000
```

Inicia la ejecución en la dirección 0x1000.

```
--exec=0x1000,steps=100
```

Inicia la ejecución en la dirección 0x1000 y ejecuta como máximo 100 instrucciones antes de devolver el control.

2.3.5. **-rom, -r**

Carga una imagen de ROM en el emulador antes de la secuencia de reset del procesador. La imagen de ROM contiene toda la información requerida por el emulador, incluyendo la tabla de vectores de interrupción y el punto de entrada de reset. Su ubicación dentro del espacio de direcciones del emulador está fijada por la arquitectura del sistema y no se puede sobrescribir.

Gramática

```
--rom=FILE
```

Elemento	Descripción
FILE	Ruta a la imagen de ROM.

Comportamiento por defecto

No se carga ninguna imagen de ROM a menos que se especifique explícitamente la opción **-rom**. Cuando está presente, la imagen de ROM se mapea en la región de ROM antes de que comience la secuencia de reset del procesador. El procesador luego realiza su inicialización de reset normal y comienza la ejecución desde el vector de reset contenido en la imagen de ROM. La dirección de ejecución inicial puede ser sobrescrita por la opción **-exec** para fines de desarrollo y depuración.

Ejemplos

```
--rom=boot.bin
```

Carga la imagen de ROM `boot.bin` antes de iniciar el emulador.

2.3.6. **-kernel, -k**

Especifica el tamaño de la región de memoria del kernel. La región de memoria del kernel está vinculada al espacio de direcciones del kernel y está disponible para software privilegiado.

Gramática

```
--kernel=SIZE
```

Elemento	Descripción
SIZE	Tamaño de la región de memoria del kernel. Los sufijos soportados son K, M y G.

Comportamiento por defecto

No se asigna memoria del kernel a menos que se especifique explícitamente la opción **-kernel**. El tamaño especificado debe estar alineado por página (4 KiB) y no debe exceder los 2 MiB. Las restricciones de arquitectura adicionales se describen en la sección Mapa de memoria.

Ejemplos

```
--kernel=64K
```

Asigna una región de memoria del kernel de 64 KiB.

2.3.7. `-memory, -m`

Especifica el tamaño de la región de memoria de usuario. La región de memoria de usuario está vinculada al espacio de direcciones de usuario y proporciona la RAM principal disponible para las aplicaciones de usuario.

Gramática

```
--memory=SIZE
```

Elemento	Descripción
SIZE	Tamaño de la región de memoria de usuario. Los sufijos soportados son K, M y G.

Comportamiento por defecto

Si se omite la opción, se asigna una región de memoria de usuario de 4 KiB. El tamaño especificado debe estar alineado por página (4 KiB) y no debe exceder los 2 MiB. Las restricciones de arquitectura adicionales se describen en la sección Mapa de memoria.

Ejemplos

```
--memory=256K
```

Asigna una región de memoria de usuario de 256 KiB.

2.3.8. `-config, -c`

Especifica el archivo de configuración que se cargará durante la inicialización del emulador. El archivo de configuración debe contener una sintaxis TOML válida. La extensión del nombre de archivo no es significativa.

Gramática

```
--config=FILE
```

Elemento	Descripción
FILE	Ruta al archivo de configuración.

Comportamiento por defecto

Si no se especifica la opción `-config`, el emulador busca `rtm32.toml` en el directorio de trabajo actual. Si el archivo de configuración especificado no se puede abrir o contiene una sintaxis TOML inválida, el emulador finaliza con un error de configuración.

Ejemplos

```
--config=board.cfg
```

Carga la configuración desde `board.cfg`. Aunque el archivo utiliza una extensión `.cfg`, su contenido debe cumplir con la especificación TOML.

Parte II

Arquitectura del procesador STX4

3. El procesador STX4

3.1. Modelo de Ejecución

La arquitectura TM32 está diseñada bajo un paradigma de tipo RISC (*Reduced Instruction Set Computer*) con una longitud de palabra de 32 bits. Desde la perspectiva del programador y del usuario del emulador, el procesador se comporta bajo un **modelo de ejecución secuencial y de ciclo único** (*single-cycle*).

Esto implica que cada instrucción se busca, decodifica, ejecuta y retira de manera atómica antes de comenzar el procesamiento de la siguiente instrucción. La semántica de ejecución garantiza que el estado de la máquina se actualiza de forma síncrona al final de cada ciclo de instrucción, eliminando complejidades visibles al software tales como riesgos de datos (*data hazards*), riesgos de control (*control hazards*) o la necesidad de ranuras de retardo de salto (*branch delay slots*). La codificación de las instrucciones se clasifica en tres formatos principales R, I y J detallados en el Capítulo 6.

El procesador denominado STX4 es una implementación de la arquitectura TM32.

3.2. Estado Arquitectónico

El estado arquitectónico representa el conjunto mínimo de elementos de almacenamiento (registros y memoria) que definen unívocamente la situación del sistema en un instante de tiempo dado. En el procesador STX4, este estado está constituido por:

1. **Banco de Registros de Propósito General (GPR):** Un conjunto de registros de 32 bits de ancho encargados de almacenar operandos y resultados intermedios.
2. **Program Counter (PC):** Registro de 32 bits que apunta a la dirección de memoria de la instrucción en curso.
3. **Banco de Registros de Funciones Especiales (SFR):** Un espacio de almacenamiento interno destinado a la configuración, el control y la gestión de excepciones y estado del procesador.
4. **Memoria Principal (M):** Un espacio físico direccionable por bytes donde residen las instrucciones y los datos del sistema.

3.3. Banco de Registros

La arquitectura RISC ortogonal TM32 de 32 bits cuenta con un banco de 32 registros de propósito general (GPR) para el usuario numerados de 0 a 31 como se observa en la tabla 3.1 donde se exhibe la designación mnemotécnica (*mnemonic*) usada por el ensamblador (columna ABI) y el propósito o convención que deberá seguir el programador y/o la herramienta de desarrollo que haga uso de los mismos.

El registro cero (**\$zero** o **\$0**) siempre contiene el valor 0. Está cableado en el hardware, así que no se puede modificar, aunque puede usarse como destino en cualquier instrucción.

Reg.	ABI	Convención
\$0	\$zero	Cableado a cero ¹
\$1	\$ra	Dirección de retorno
\$2,\$3	\$k0,\$k1	Reservados
\$4,...,\$7	\$lr0,...,\$lr3	Registros de enlace
\$8,...,\$13	\$a0,...,\$a5	Argumentos de función
\$14,...,\$19	\$t0,...,\$t5	Temporarios
\$20,...,\$27	\$s0,...,\$s7	Almacenables
\$28	\$fp	Puntero de marco
\$29	\$gp	Puntero global
\$30	\$sp	Puntero de pila
\$31	\$at	Ensamblador

Tabla 3.1: Uso y convención de los registros RTM32

El registro `$at` (*assembler temporary*) es muy utilizado por el compilador de assembly para almacenar valores temporales que se utilizan dentro de pseudo-instrucciones. No se preserva entre function calls.

Los registros `$a0,...,$a5` (*argument*) se usan para pasar argumentos a funciones. Si el número de argumentos es mayor se debe utilizar la pila. No se preservan entre function calls al igual que los registros temporales `$t0,...,$t5` (*temporaries*) que son usados por el compilador o por el programador de assembly para almacenar valores intermedios. Los registros `$a0` y `$a1` tienen como alias `$v0` y `$v1` y se pueden usar para retornar valores desde funciones. No se preservan entre llamadas (*function calls*). Normalmente solo se usa el primero pero es posible retornar valores de 64 bits mediante el uso del segundo para contener los cuatro bytes más significativos. Los registros `$s0,...,$s7` (*Saved Temporary*) se usan para almacenar valores de mayor duración y sí se preservan entre function calls.

Los registros de enlazado `$lr0,...,$lr3` (*link register*) requieren una especial mención. Pueden ser utilizados por la instrucción JALX para almacenar múltiples retornos en aplicaciones como corrutinas. No se preservan y por lo tanto pueden ser reutilizados cuando no están en uso.

Los registros `$k0,$k1` están reservados para uso del kernel del sistema operativo. Pueden cambiar de forma impredecible en cualquier momento porque son utilizados por los interrupt handlers.

Los registros de puntero (*pointer registers*) son una familia de registros que contiene direcciones de memoria, a lo que se les asigna funciones muy particulares. Todos ellos se preservan entre function calls y no todos son de uso obligatorio, por lo que pueden cumplir una función secundaria con otro alias. Por ejemplo `$fp` tiene como alias `$s8` y `$gp` alias `$s9`. Es importante entender que mientras se los use como otros registros no podrán ser punteros y viceversa.

A continuación detallamos la función asignada a cada uno de ellos:

- Puntero global `$gp` (*Global Pointer*): normalmente guarda un puntero al área de datos globales (para que pueda accederse usando memory offset addressing).
- Puntero de pila `$sp` (*Stack Pointer*): se usa para almacenar la dirección a donde apunta la pila actualmente.

- Puntero de marco **\$fp** (*Frame Pointer*): se usa para almacenar la dirección base del marco de pila (*Stack Frame*).
- Dirección de retorno **\$ra** (*Return Address*): almacena la dirección de retorno (la ubicación del programa a la que una función debe volver). Este puntero es usado comúnmente para las instrucciones **JAL** y **JALR**.

Todos los registros mencionados son utilizados regularmente en la CPU y accesible a través de las instrucciones pero hay otros registros como el contador de programa **\$pc** (*program counter*) que es manipulado exclusivamente por la CPU e indirectamente por las instrucciones de cambio de flujo.

3.4. Registros de Funciones Especiales

Los registros de funciones especiales **SFR** (*special function registers*) permiten inspeccionar o setear distintas condiciones de la CPU y se acceden en forma indirecta mediante instrucciones específicas como **CFS** y **CTS**. La CPU STX4 tiene 7 registros especiales que detallamos a continuación:

- Palabra de estado del programa **\$psw** (*Program Status Word*): almacena el estado de ejecución del procesador, incluyendo las banderas aritméticas, el nivel de privilegio y el estado global de las interrupciones.
- Registro de causa de la excepción **\$ecr** (*Exception Cause Register*): contiene el código que identifica la excepción o interrupción que provocó la transferencia de control al manejador correspondiente.
- Contador de programa de la excepción **\$epc** (*Exception Program Counter*): almacena la dirección de la instrucción cuya ejecución fue interrumpida por la excepción o interrupción, permitiendo reanudar la ejecución cuando corresponda.
- Registro de estado de la excepción **\$esr** (*Exception Status Register*): almacena el valor del **\$psw** al momento en que se produce una excepción o interrupción, permitiendo restaurar su valor cuando corresponda.
- Dirección virtual incorrecta **\$bva** (*Bad Virtual Address*): almacena la dirección virtual que originó una excepción de acceso a memoria.
- Registro base vectorizado **\$vbr** (*Vector Base Register*): contiene la dirección base de la tabla de vectores de excepción e interrupción. Cuando se resetea el procesador este registro se carga con el valor **0xF0000000**.
- Registro de identificación de la CPU **\$pir** (*Processor Identification Register*): contiene un conjunto de bits que identifican el procesador, su familia y si posee características especiales. Está cableado de fábrica y su valor no puede alterarse.

3.4.1. El PSW

Este registro está compuesto por un conjunto de bits detallado en la figura 3.1 que representan el estado del procesador. Los que permiten escritura determinan ciertas configuraciones posibles como la habilitación de interrupciones o la ejecución de traps en modo privilegiado. A continuación se detalla el comportamiento y la función semántica de cada uno de los bits de control y banderas de condición que conforman el registro **\$psw**:

**Figura 3.1:** Formato del registro PSW

Bit	Nombre	Descripción y Funcionamiento
M	Modo de Ejecución	Determina el nivel de privilegio actual del procesador. • 1: Modo Kernel (Privilegiado). Permite la ejecución de todas las instrucciones del ISA, incluyendo aquellas de control de sistema, manipulación de registros de control y acceso irrestricto al mapa de memoria. • 0: Modo Usuario (No Privilegiado). Restringe las operaciones sensibles de máquina. Cualquier intento de ejecutar instrucciones de control en este modo gatilla de forma automática un <i>trap</i> por violación de privilegios.
I	Habilitación de Interrupciones	Control global sobre la aceptación de peticiones de interrupción de hardware externas (enmascarables). • 1: Habilitadas. El procesador responderá y derivará el control al manejador de interrupciones correspondiente al recibir una petición. • 0: Deshabilitadas. Las interrupciones externas quedan bloqueadas temporalmente en el hardware del procesador.
T	Habilitación de Traps	Controla la captura de excepciones generadas por software o errores de ejecución interna cuando el sistema opera en el nivel de mayor privilegio. • 1: Habilitado. Permite que se generen y capturen <i>traps</i> de manera de forma controlada y segura incluso cuando el procesador se encuentra ejecutando en Modo Kernel. • 0: Deshabilitado.
Z	Bandera de Cero	Indica si el resultado de la última operación en la ALU fue idénticamente nulo. Se evalúa en 1 si el resultado es cero; de lo contrario, se limpia a 0.
N	Bandera de Negativo	Refleja el bit más significativo (bit de signo) del resultado de la última operación aritmética. Se establece en 1 si el resultado es negativo; de lo contrario, toma el valor 0.
C	Bandera de Acarreo	Se establece en 1 si se produce un acarreo de salida (<i>carry-out</i>) o un préstamo (<i>borrow</i>) más allá del bit más significativo en operaciones aritméticas de suma o resta; de lo contrario, se limpia a 0.
O	Bandera de Desbordamiento	Se activa en 1 si ocurre un desbordamiento aritmético en operaciones con signo bajo la representación de complemento a dos (por ejemplo, cuando la suma de dos valores de igual signo produce un resultado con signo opuesto); de lo contrario, permanece en 0.

Tabla 3.2: Descripción detallada de los bits del registro PSW

3.5. Modelo de Privilegios

El procesador STX4 implementa un modelo de seguridad dual basado en dos niveles de ejecución jerárquicos gobernados por el bit **M** del registro de estado `$psw`: el **Modo Kernel** (privilegiado, $M = 1$) y el **Modo Usuario** (no privilegiado, $M = 0$).

Toda transición hacia un entorno seguro de ejecución se gestiona a través de excepciones de software controladas mediante la instrucción `TRAP`, la cual almacena la dirección de retorno en el registro `$epc` aplicando un desfase fijo de 4 bytes sobre el program counter actual

($EPC = PC + 4$) y desvía el flujo del programa hacia el vector de servicio correspondiente en la dirección de memoria apuntada por la expresión $\$vbr + (param \ll 2)$. El retorno desde el contexto privilegiado hacia el hilo de ejecución original se efectúa de forma atómica mediante la instrucción **RFT**, la cual restaura el contador de programa desde el registro $\$epc$ restableciendo además los contextos de estado guardados en $\$esr$ hacia el $\$psw$.

3.5.1. Estado de Reset

La activación de la señal física de reset fuerza al procesador a un estado inicial predecible e inmune a condiciones de carrera de software. El estado arquitectónico del procesador tras un evento de reset (*Cold Start*) se establece de la siguiente manera:

1. **Program Counter (PC):** El contador de programa es forzado a inicializarse en la dirección de memoria física $0xF0000000$ (definida como el vector de inicio de la máquina o Vector 0), lugar donde debe residir el firmware o cargador de arranque primario del sistema.
2. **Registro de Estado (\$psw):** Se inicializa con los bits de control configurados para permitir la inicialización segura del software de sistema:
 - El bit **M** se establece en 1 (Modo Kernel), otorgando acceso privilegiado completo al hardware.
 - El bit **I** se establece en 0 (Interrupciones deshabilitadas), previniendo la interferencia de periféricos externos antes de que se configuren las tablas de vectores.
 - El bit **T** se establece en 1 (Traps habilitados), permitiendo capturar excepciones de software generadas de forma síncrona en el arranque.
3. **Banco de Registros de Propósito General:** Todos los registros del banco ($R[0]$ a $R[31]$) son borrados de manera síncrona por hardware, garantizando un estado inicial limpio con valor $0x00000000$ para todos ellos al comenzar la ejecución.
4. **Registro Base Vectorizado (\$vbr):** Se inicializa cargando por defecto el valor de dirección $0xF0000000$, alineando el espacio inicial de excepciones con el segmento de memoria de arranque.

3.6. Tipos de Datos

La arquitectura RTM32 es una máquina orientada a palabras de 32 bits que opera fundamentalmente sobre enteros y direcciones de memoria. A continuación, se detallan los tipos de datos fundamentales soportados por el hardware, su representación binaria y el tratamiento que el procesador aplica a los operandos inmediatos y las direcciones de memoria.

3.6.1. Tipos de Datos Fundamentales y Tamaños

El procesador define tres tamaños de datos básicos para la manipulación de información en registros y memoria:

- **Palabra (*Word*)**: El tipo de dato nativo de la arquitectura, con un ancho de 32 bits (4 bytes). Los registros de propósito general (R[0] a R[31]) y las operaciones de la ALU operan de manera nativa en este formato.
- **Media Palabra (*Half-word*)**: Datos con un ancho de 16 bits (2 bytes). Utilizado principalmente en transferencias de memoria estructuradas y operaciones de empaquetado.
- **Byte**: La unidad mínima de direccionamiento de la memoria, con un ancho de 8 bits (1 byte).

3.6.2. Representación de Enteros e Interpretación

Los tipos de datos numéricos se interpretan bajo dos esquemas de codificación binaria dependiendo de la instrucción ejecutada (por ejemplo, diferencias entre BLT y BLTU, o SLTI y SLTIU):

Representación Sin Signo (*Unsigned*)

Un entero sin signo de n bits permite representar valores en el rango $[0, 2^n - 1]$. Para una palabra completa de 32 bits, el rango corresponde a $[0, 4\,294\,967\,295]$. Su valor decimal se evalúa mediante la sumatoria estándar de pesos binarios:

$$U(X) = \sum_{i=0}^{n-1} x_i \cdot 2^i$$

Representación Con Signo (*Signed*)

Los enteros con signo se representan exclusivamente mediante el formato de **complemento a dos**. El bit más significativo (x_{n-1}) actúa como bit de signo. El rango de valores representables para una palabra de 32 bits bajo este esquema es $[-2^{31}, 2^{31} - 1]$, equivalente a $[-2\,147\,483\,648, 2\,147\,483\,647]$. El valor decimal se calcula como:

$$S(X) = -x_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} x_i \cdot 2^i$$

Las instrucciones de carga de memoria (LH, LHU, LB, LBU) realizan la transferencia desde la memoria hacia un registro destino aplicando extensión de signo (añadiendo copias de x_{15} o x_7 en los bits más significativos del registro de 32 bits) o extensión de cero, de acuerdo con la semántica del mnemónico.

3.6.3. Valores Inmediatos y Extensión de Tipo

Las instrucciones de formato I codifican operandos inmediatos dentro de la propia instrucción. Para adaptar estos valores al ancho nativo de la ALU (32 bits), el procesador implementa dos operadores matemáticos de extensión denotados como $E_x(y, z)$ y de concatenación $C_x(y, z)$:

1. **Extensión de Signo o Cero ($E_x(y, z)$)**: Extiende un valor y de x bits hasta completar los 32 bits utilizando un bit de relleno $z \in \{S, 0, 1\}$.

- **Extensión con Signo ($E_x(y, S)$):** Los $32 - x$ bits más significativos del operando final se rellenan con el bit de signo del inmediato original (bit y_{x-1}). Se utiliza en instrucciones aritméticas con signo como ADDI o desplazamientos de memoria como LW.
 - **Extensión con Cero ($E_x(y, 0)$):** Los bits restantes se rellenan con 0. Utilizado en operaciones lógicas como ANDI u ORI.
2. **Concatenación de Bits ($C_x(y, z)$):** Utilizada para construir constantes en la parte alta de un registro (instrucciones tipo LCI, ANDH, ORH, XORH). Concatena los x bits del inmediato y en los bits más significativos de la palabra, preservando o modificando los bits menos significativos provenientes del registro fuente z .

3.6.4. Representación de Direcciones de Memoria

La arquitectura RTM32 utiliza un espacio de direccionamiento plano y lineal de 32 bits, lo que permite un límite teórico de acceso a memoria de 4 GiB (2^{32} bytes).

- **Direcciones Relativas al PC (RA_x):** Los saltos condicionales e incondicionales calculan su destino de forma relativa al registro PC. El procesador opera en direccionamiento por palabras de 32 bits para instrucciones, calculando la dirección destino como:

$$RA_x(y) = PC + 4 + E_x(4y, S)$$

Multiplicar el inmediato por 4 desplaza implícitamente el operando 2 bits a la izquierda, optimizando el rango de salto bajo la premisa de que las instrucciones siempre se encuentran alineadas a palabra en memoria.

- **Direcciones Absolutas (AA_x):** Saltos indirectos calculados como base más desplazamiento:

$$AA_x(rs, imm) = R[rs] + E_x(4imm, S)$$

- **Dirección Efectiva de Datos (EA y EAX):** Para accesos de lectura/escritura a memoria de datos, la dirección efectiva de transferencia se determina mediante la suma del registro base y el inmediato extendido con signo ($EA = R[rs] + E_{17}(imm, S)$) o mediante la suma indexada de dos registros en instrucciones de tipo indexado ($EAX(rs, rd) = R[rs] + R[rd]$).

3.6.5. Concepto de Desbordamiento (*Overflow*)

El desbordamiento aritmético ocurre exclusivamente en operaciones con signo cuando el resultado matemático de una operación excede el rango de representación del formato de complemento a dos de 32 bits (rango $[-2^{31}, 2^{31} - 1]$).

El procesador detecta esta condición mediante hardware dedicado en la ALU y actualiza de manera síncrona la bandera de desbordamiento **O** (*Overflow*) en el registro de estado \$psw. Conceptualmente, para una operación de suma $Z = X + Y$, la bandera **O** se activa si y solo si:

- Se suman dos operandos con signo positivo ($x_{31} = 0, y_{31} = 0$) y el resultado tiene signo negativo ($z_{31} = 1$).

- Se suman dos operandos con signo negativo ($x_{31} = 1, y_{31} = 1$) y el resultado tiene signo positivo ($z_{31} = 0$).

En operaciones sin signo, las condiciones de rebase de rango se registran únicamente a través de la bandera de acarreo **C** (*Carry*), evitando alterar la bandera de desbordamiento aritmético con signo.

Parte III

Arquitectura del Set de Instrucciones

4. Notación

4.1. Convenciones generales

A lo largo de este manual las instrucciones de RTM32 se describen mediante un lenguaje matemático formal. Las funciones, operadores y convenciones definidas en este capítulo forman parte de la especificación arquitectónica del ISA y se utilizarán en los capítulos posteriores sin volver a redefinirse.

Salvo que se indique explícitamente lo contrario, todos los registros y operandos poseen el tamaño nativo de la arquitectura (32 bits). Las constantes inmediatas se representan utilizando el número de bits definido por el formato de instrucción correspondiente.

ATENCIÓN

Las funciones definidas en este capítulo no representan instrucciones de la arquitectura ni operaciones realizadas por el compilador. Constituyen una notación matemática utilizada para describir de manera precisa el comportamiento de cada instrucción del ISA.

4.2. Operaciones sobre bits

Algunas instrucciones operan únicamente sobre una parte del contenido de un registro o de un operando. Para describir estas operaciones la arquitectura utiliza la notación de selección de campos de bits.

4.2.1. Selección de bits

La expresión

$$x_{[m:n]}$$

indica el subconjunto de bits comprendidos entre las posiciones m y n , siendo $m \geq n$ y m el bit más significativo del rango. Los bits se numeran desde cero, correspondiendo el bit 0 al bit menos significativo.

Por ejempl si $x = 0x12345678$ entonces $x_{[15:0]} = 0x5678$ y $x_{[31:16]} = 0x1234$.

4.2.2. Concatenación binaria

El operador

$$x||y$$

representa la concatenación de dos secuencias binarias, donde los bits de x forman la parte más significativa del resultado y los bits de y la parte menos significativa. Este operador constituye la base de la función de concatenación $C()$ a definir.

Por ejemplo,

$$0x1234||0xABCD = 0x1234ABCD$$

4.3. Espacios de almacenamiento

Las siguientes expresiones identifican los distintos espacios de almacenamiento utilizados por la arquitectura.

$R[x]$	Registro de propósito general con índice x .
$S[x]$	Registro especial con índice x .
$M[x]$	Contenido de memoria ubicado en la dirección x .
PC	Contador de programa.
EPC	Contador de programa almacenado durante una excepción.
$\$0$	Registro constante cuyo valor es siempre cero.

Cuando una expresión utiliza índices (selección de bits) como en el ejemplo,

$$R[5]_{[15:0]}$$

representa los dieciséis bits menos significativos del registro $R[5]$.

4.4. Transformación de operandos

Las instrucciones de RTM32 utilizan distintas funciones para adaptar el tamaño de operandos inmediatos antes de participar en operaciones aritméticas, lógicas o en el cálculo de direcciones.

Estas funciones reciben como entrada un operando de x bits y producen un resultado del tamaño de palabra de la arquitectura.

4.4.1. Extensión de operandos

La función general de extensión se define como

$$E_x(y, z) \tag{4.1}$$

donde

- x representa el tamaño original del operando.
- y representa el valor a extender.
- z especifica el mecanismo de extensión.

El parámetro

$$z \in \{S, 0, 1\}$$

selecciona uno de los tres mecanismos de extensión definidos por la arquitectura.

Extensión con signo

La extensión con signo conserva el valor numérico de un operando representado en complemento a dos.

Formalmente,

$$E_x(y, S) = \underbrace{b_{x-1} \dots b_{x-1}}_{32-x} b_{x-1} \dots b_0 \quad (4.2)$$

donde b_{x-1} representa el bit de signo del operando original.

En consecuencia,

- si el bit de signo vale 0, la extensión completa con ceros;
- si el bit de signo vale 1, la extensión completa con unos.

La extensión con signo preserva el valor aritmético del operando al cambiar su tamaño de representación. Este mecanismo se utiliza para constantes inmediatas con signo, desplazamientos relativos y cargas de datos con signo.

Ejemplo 1

$$E_8(0x2A, S) = 0x0000002A$$

Como el bit de signo vale cero, los bits superiores se completan con ceros.

Ejemplo 2

$$E_8(0xF4, S) = 0xFFFFFFF4$$

En este caso el operando representa el valor decimal -12 , por lo que los bits superiores se completan con unos.

Extensión con ceros

La extensión con ceros copia el operando original completando los bits superiores con ceros.

$$E_x(y, 0) = \underbrace{00 \dots 0}_{32-x} b_{x-1} \dots b_0 \quad (4.3)$$

Este mecanismo interpreta el operando como un valor sin signo. La extensión con ceros se emplea principalmente en operaciones lógicas y en las instrucciones que manipulan datos sin signo.

Por ejemplo,

$$E_{16}(0xABCD, 0) = 0x0000ABCD$$

Extensión con unos

La extensión con unos completa los bits superiores utilizando el valor uno.

$$E_x(y, 1) = \underbrace{11 \dots 1}_{32-x} b_{x-1} \dots b_0 \quad (4.4)$$

Este mecanismo se utiliza principalmente para construir máscaras donde los bits más significativos deben permanecer activos.

⚠ ATENCIÓN

La extensión con unos no preserva el valor numérico del operando. Su finalidad es facilitar la construcción de máscaras binarias empleadas por determinadas instrucciones lógicas de la arquitectura.

Por ejemplo,

$$E_{16}(0x1234, 1) = 0xFFFF1234$$

4.4.2. Concatenación

Algunas instrucciones requieren construir un nuevo operando combinando dos valores distintos. Para ello la arquitectura define la función de concatenación

$$C_x(y, z) \tag{4.5}$$

que concatena los x bits de y como parte más significativa con los x bits menos significativos de z .

Formalmente,

$$C_x(y, z) = y_{[x-1:0]} \parallel z_{[x-1:0]}$$

donde el operador $\parallel$ representa la concatenación de secuencias binarias.

📄 NOTA

La función de concatenación permite construir constantes de 32 bits mediante una única instrucción, reutilizando parte del contenido previo de un registro. Este mecanismo es utilizado por las instrucciones LCI, ANDH, ORH y XORH.

Ejemplo Si

$$R[\$t0] = 0x56789ABC$$

entonces

$$C_{16}(0x1234, R[\$t0]) = 0x12349ABC$$

ya que los dieciséis bits superiores provienen del operando inmediato, mientras que los dieciséis bits inferiores conservan el contenido original del registro.

Constantes ubicadas en la mitad superior Con frecuencia resulta conveniente representar constantes de 32 bits cuyo valor inmediato ocupa la mitad superior de la palabra y cuya mitad inferior está formada por un valor constante. Para simplificar la notación se definen las siguientes funciones auxiliares:

$$H_{16}(imm, 0) \equiv C_{16}(imm, \$0)$$

$$H_{16}(imm, 1) \equiv C_{16}(imm, \neg\$0)$$

donde $\$0$ representa el registro constante con valor $0x00000000$ y $\neg\$0$ denota conceptualmente un registro con valor $0xFFFFFFFF$. Estas funciones son únicamente alias de la operación de concatenación y se utilizan para expresar de forma más clara las operaciones lógicas inmediatas sobre la mitad superior del registro.

4.5. Interpretación de operandos

Salvo que se indique explícitamente lo contrario, todos los operandos de la arquitectura se interpretan como enteros representados en complemento a dos.

Para aquellas instrucciones que requieren interpretar un operando como un valor sin signo se utiliza la función

$$U(x) \tag{4.6}$$

la cual indica que el operando debe considerarse como un entero positivo representado mediante aritmética modular de 32 bits.

NOTA

La función $U()$ modifica únicamente la interpretación matemática del operando. No altera el patrón binario almacenado en el registro.

Por ejemplo,

$$0xFFFFFFFF = -1$$

cuando se interpreta como entero con signo, mientras que

$$U(0xFFFFFFFF) = 4294967295$$

cuando se interpreta como entero sin signo.

Esta notación se emplea principalmente en las instrucciones de comparación, multiplicación y división sin signo.

4.6. Cálculo de direcciones

Las instrucciones que acceden a memoria o modifican el flujo de ejecución obtienen la dirección de destino mediante funciones arquitectónicas especializadas. Estas funciones forman parte de la especificación formal del ISA y serán utilizadas en la descripción de todas las instrucciones correspondientes.

4.6.1. Dirección efectiva

Las instrucciones de acceso a memoria con desplazamiento inmediato utilizan la función

$$EA(rs, imm) = R[rs] + E_{17}(imm, S) \quad (4.7)$$

donde $R[rs]$ proporciona la dirección base y el operando inmediato representa un desplazamiento relativo expresado en bytes.

Por ejemplo. si

$$R[\$sp] = 0x1000$$

entonces

$$EA(\$sp, 8) = 0x1008.$$

4.6.2. Dirección efectiva extendida

Las instrucciones de acceso a memoria del formato R utilizan dos registros para calcular la dirección efectiva.

La función correspondiente se define como

$$EAX(rs, rd) = R[rs] + R[rd] \quad (4.8)$$

Este mecanismo elimina la necesidad de utilizar un operando inmediato y permite indexar estructuras de datos utilizando registros.

Por ejemplo, si

$$R[\$s0] = 0x1000$$

y

$$R[\$t0] = 0x0010$$

entonces

$$EAX(\$s0, \$t0) = 0x1010.$$

4.6.3. Dirección relativa al contador de programa

Las instrucciones de salto relativo utilizan la función

$$RA_x(imm) = PC + 4 + E_x(4 \cdot imm, S) \quad (4.9)$$

donde el subíndice x indica el tamaño efectivo del operando inmediato empleado por la instrucción correspondiente.

El desplazamiento inmediato se expresa en palabras y posteriormente se convierte a bytes multiplicándolo por cuatro.

NOTA

La referencia utilizada por la función $RA()$ corresponde siempre a la dirección de la siguiente instrucción ($PC+4$), independientemente del tipo de salto.

4.6.4. Dirección absoluta relativa a un registro

Las instrucciones de salto indirecto utilizan la función

$$AA_x(rs, imm) = R[rs] + E_x(4 \cdot imm, S) \quad (4.10)$$

donde el desplazamiento inmediato también se expresa en palabras y se convierte a bytes antes de realizar la suma.

A diferencia de la función $RA()$, la dirección base no proviene del contador de programa sino del contenido de un registro de propósito general.

4.6.5. Entrada de la tabla de vectores

La arquitectura define la función

$$TV(param) = param \ll 2 \quad (4.11)$$

que calcula el desplazamiento correspondiente a una entrada **param** dentro de la tabla de vectores de excepción. Cada entrada ocupa una palabra de memoria (4 bytes), razón por la cual el índice se multiplica por cuatro mediante un desplazamiento de dos bits hacia la izquierda.

Por ejemplo, si $param = 5$ entonces

$$TV(5) = 20 = 0x14.$$

NOTA

La función $TV()$ calcula únicamente el desplazamiento de una entrada dentro de la tabla de vectores. La dirección base de dicha tabla es proporcionada por la implementación de la arquitectura o por el sistema operativo.

4.7. Nomenclatura de operandos inmediatos

Las instrucciones de RTM32 emplean operandos inmediatos de distintos tamaños, dependiendo del formato de instrucción y de la operación realizada. Para simplificar la descripción de la ISA, este manual utiliza una convención de nombres donde el identificador del operando refleja directamente la cantidad relativa de bits disponibles para representar el valor inmediato. La Tabla 4.1 resume la nomenclatura empleada a lo largo del manual.

La longitud efectiva de cada operando depende del formato de instrucción en el que aparece y se encuentra determinada por la codificación binaria del ISA. Por este motivo, los

Notación	Significado	Relativo	Tamaño
elimm	<i>Extra Long Immediate</i>	Mayor	25 bits
vlimm	<i>Very Long Immediate</i>		24 bits
llimm	<i>Long Long Immediate</i>		20 bits
limm	<i>Long Immediate</i>		19 bits
imm	<i>Immediate</i>		17 bits
simm	<i>Short Immediate</i>	Menor	16 bits

Tabla 4.1: Jerarquía de operandos inmediatos

nombres anteriores constituyen una nomenclatura relativa y no representan un número fijo de bits.

Esta convención permite describir el comportamiento de las instrucciones de forma independiente de la distribución concreta de campos dentro de cada formato de instrucción, manteniendo una notación uniforme en toda la especificación arquitectónica.

NOTA

Los prefijos utilizados siguen una jerarquía estrictamente decreciente:

$$\text{vlimm} > \text{llimm} > \text{limm} > \text{imm} > \text{simm}$$

donde cada nivel dispone de un campo inmediato menor o igual que el anterior. La cantidad exacta de bits de cada operando se define en el capítulo dedicado a los formatos de instrucción.

5. Modos de Direccionamiento

5.1. Consideraciones generales

Los modos de direccionamiento definen la forma en que una instrucción obtiene la ubicación de sus operandos. Dependiendo de la operación, éstos pueden encontrarse en registros de propósito general, formar parte de la propia instrucción o calcularse dinámicamente a partir de uno o más registros del procesador.

En RTM32 el modo de direccionamiento no se codifica mediante un campo específico dentro de la instrucción. En su lugar, queda determinado de manera implícita por el **opcode** y por el formato de instrucción utilizado. Este enfoque simplifica el proceso de decodificación, reduce la complejidad del hardware de control y permite mantener un formato de instrucción uniforme de 32 bits para toda la arquitectura.

La arquitectura define cinco modos fundamentales de direccionamiento:

- Direccionamiento mediante registros.
- Direccionamiento inmediato.
- Direccionamiento indexado.
- Direccionamiento relativo al contador de programa.
- Direccionamiento indirecto.

Cada uno de ellos responde a necesidades diferentes y ha sido diseñado para optimizar las operaciones más frecuentes realizadas por el procesador.

5.2. Direccionamiento mediante registros

El direccionamiento mediante registros (*Register Addressing*) utiliza como operandos el contenido de los registros de propósito general. En este modo la instrucción únicamente codifica los índices de los registros involucrados; los datos propiamente dichos nunca forman parte de la palabra de instrucción.

Las instrucciones de formato R emplean este mecanismo para la mayoría de las operaciones aritméticas, lógicas, desplazamientos, comparaciones, multiplicación, división y acceso a registros especiales.

Generalmente los campos **rs** y **rt** identifican los registros fuente, mientras que **rd** selecciona el registro destino. Algunas familias reinterpretan parcialmente estos campos, aunque el principio de funcionamiento permanece inalterado.

Al encontrarse todos los operandos disponibles dentro del banco de registros, este modo evita accesos adicionales a memoria y constituye el mecanismo de direccionamiento de menor latencia de la arquitectura.

```
ADD  $t0, $t1, $t2
AND  $s0, $s1, $s2
SLTU $a0, $a1, $a2
MUL  $v0, $v1, $t0
```

El direccionamiento mediante registros constituye el mecanismo preferido para el procesamiento intensivo de datos. Los compiladores intentan mantener las variables más utilizadas en registros para minimizar los accesos a memoria y mejorar el rendimiento del programa.

5.3. Direccionamiento inmediato

En el direccionamiento inmediato (*Immediate Addressing*) uno de los operandos se encuentra codificado directamente dentro de la instrucción. De este modo la CPU dispone inmediatamente del valor constante sin necesidad de realizar accesos adicionales a memoria.

Dependiendo de la instrucción, la constante inmediata puede representar un valor aritmético, una máscara lógica, un desplazamiento o una parte de una constante de mayor tamaño.

Antes de participar en la operación correspondiente, el operando inmediato se transforma utilizando las funciones definidas en la sección 4.4. Según la instrucción, la arquitectura puede aplicar extensión con signo, extensión con ceros, extensión con unos o concatenación.

Este modo de direccionamiento es utilizado por todas las instrucciones de operación inmediata del formato I.

```
ADDI $t0,$t1,-4
ANDI $s0,$s0,0x00FF
ORI  $a0,$a0,0x10
LCI  $t2,$t2,0x1234
```

NOTA

Las transformaciones de operandos inmediatos se describen mediante las funciones `E()` y `C()`, definidas en las secciones 4.4.1 y 4.4.2 respectivamente. Estas funciones forman parte de la especificación formal de la arquitectura y se referenciarán a lo largo de todo el manual.

5.4. Direccionamiento indexado

El direccionamiento indexado (*Base+Offset Addressing*) constituye el mecanismo utilizado por todas las instrucciones de acceso a memoria del formato I. En este modo la instrucción especifica un registro base y un desplazamiento inmediato a partir del cual la CPU obtiene la dirección del operando.

Durante la ejecución la dirección efectiva se calcula aplicando la función `EA()`, definida en la sección 4.6.1. Como el desplazamiento inmediato se representa en complemento a dos, el acceso puede realizarse tanto hacia direcciones superiores como inferiores respecto del registro base.

Este mecanismo resulta especialmente adecuado para acceder a variables locales, registros de activación, estructuras de datos, arreglos y pilas, permitiendo recorrer regiones completas de memoria modificando únicamente el valor del registro base. Todas las instrucciones de carga y almacenamiento utilizan este modo de direccionamiento.

```
LW  $t0,8($sp)
SW  $t1,-4($sp)
LH  $a0,2($s0)
```

```
LBU $v0, 0($a1)
SB $t2, 15($fp)
```

⚠ ATENCIÓN

El cálculo de la dirección efectiva no garantiza que el resultado satisfaga las restricciones de alineación requeridas por la instrucción. Cuando una operación accede a palabras o medias palabras, la dirección obtenida mediante `EA()` deberá encontrarse correctamente alineada. En caso contrario la CPU genera una excepción de alineación.

5.5. Direccionamiento relativo al contador de programa

El direccionamiento relativo al contador de programa (*PC-Relative Addressing*) se utiliza para las instrucciones de transferencia de control cuyo destino depende de la posición actual del programa.

En este modo la instrucción codifica únicamente un desplazamiento relativo. Durante la ejecución la CPU calcula la dirección de destino mediante la función `RA()`, definida en la sección 4.6.1, utilizando como referencia la dirección de la siguiente instrucción (`PC+4`).

Este mecanismo permite generar código independiente de la posición de carga (*Position Independent Code*), ya que el destino del salto depende únicamente del valor actual del contador de programa y no de direcciones absolutas.

Las instrucciones de salto condicional del formato I y las instrucciones de salto del formato J utilizan este modo de direccionamiento.

```
BEQ $t0, $t1, label
BNE $a0, $zero, loop
J init
JAL function
```

📄 NOTA

La utilización de desplazamientos relativos facilita la relocación del código ejecutable y reduce el tamaño de los operandos necesarios para representar direcciones de salto.

5.6. Direccionamiento indirecto

El direccionamiento indirecto (*Register-Relative Addressing*) obtiene la dirección de destino a partir del contenido de un registro base y un desplazamiento inmediato. A diferencia del direccionamiento relativo al contador de programa, la referencia utilizada durante el cálculo proviene del contenido de un registro de propósito general.

La dirección de destino se obtiene mediante la función `AA()`, definida en la sección 4.6.4. Este mecanismo permite realizar transferencias dinámicas de control cuya dirección únicamente puede conocerse durante la ejecución del programa.

Las instrucciones `JR`, `JALR` y `JALRX` utilizan este modo de direccionamiento.

```
JR $t0, 8
```

```
JALR  $s0,-2
JALRX $lr1,handler
```

Las variantes con enlace almacenan previamente la dirección de retorno en el registro correspondiente antes de efectuar la transferencia de control, permitiendo implementar llamadas a procedimientos y retornos posteriores.

NOTA

El direccionamiento indirecto resulta fundamental para implementar llamadas mediante punteros a función, tablas de despacho, máquinas virtuales, intérpretes y otras estructuras donde el destino del salto no puede conocerse durante la compilación.

5.7. Resumen

La Tabla 5.1 resume los modos de direccionamiento soportados por la arquitectura RTM32 y las familias de instrucciones que los emplean.

Modo	Referencia	Instrucciones
Mediante registros	Banco de registros	Formato R
Inmediato	Constante codificada	Operaciones inmediatas
Indexado	Registro base + desplazamiento	Cargas y almacenamientos
Relativo al PC	PC + desplazamiento	Branch, J, JAL, JALX
Indirecto	Registro base + desplazamiento	JR, JALR, JALRX

Tabla 5.1: Resumen de los modos de direccionamiento de RTM32

6. Formatos de Instrucción

6.1. Consideraciones Generales

Las instrucciones constan de un ancho fijo de 32 bits y se decodifican mediante campos bit a bit estructurados en cuatro tipos llamados: R, I y J. Todas las instrucciones comparten el mismo campo llamado **opcode** (código de operación) formado por los cinco bits más significativos (bits 31 a 27). El valor del opcode identifica al tipo de instrucción como se muestra en la tabla B.1. No todos los opcodes son válidos. Los valores de opcode no asignados se consideran codificaciones reservadas. Su ejecución genera una excepción de instrucción inválida. Durante la descripción de los formatos, los bits se numeran desde el bit 0 (menos significativo) hasta el bit 31 (más significativo). Las figuras representan las instrucciones con el bit más significativo a la izquierda.

Para simplificar la descripción de las instrucciones se utilizará la notación definida en el capítulo [Notación](#) que se resume en la tabla A.1.

6.2. Formato R-Type

El formato tipo R (*register*) que se muestra en la figura 6.1 se utiliza principalmente para operaciones aritméticas, lógicas, desplazamientos, multiplicación, división y operaciones especiales del sistema.

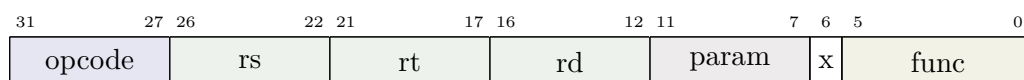


Figura 6.1: Formato tipo R

Cómo todas comparten el mismo opcode, la CPU las diferencia por el campo **func** o función (*function*) que se encuentra en los 6 bits menos significativos (bit 0 al 5). En la tabla B.2 se provee una lista de las instrucciones tipo R y sus valores de función asociados.

Los campos **rs**, **rt** y **rd** especifican registros (de allí el nombre de tipo R), haciendo referencia al primer operando fuente (*source register*), al segundo operando fuente (*temporal register*) y a el registro destino (*destination register*). El bit 6 identificado como **x** está reservado y debe contener 0¹.

6.2.1. Campo param

El campo **param** (bits 12 a 8) es un campo paramétrico (*parameter field*) de propósito múltiple de 5 bits cuyo significado depende de la instrucción. Según la instrucción, puede representar un desplazamiento inmediato, un índice de registro especial o un índice dentro de la tabla de vectores de excepción. Sus usos se especifican a continuación:

- Desplazamientos inmediatos (**SLL**, **SRL**, **SRA**): **param** contiene la cantidad de bits a desplazar (0–31).

¹Todo campo reservado deberá escribirse con cero y deberá ignorarse durante la lectura, salvo que se indique explícitamente lo contrario.

- Acceso a registros especiales (CFS, CTS): **param** contiene el índice del registro especial.
- Manejo de excepciones (TRAP): **param** contiene el índice de la entrada correspondiente dentro de la tabla de vectores de excepción.

Cuando una instrucción no utiliza el campo **param**, este deberá codificarse con el valor cero para garantizar la compatibilidad con futuras extensiones de la arquitectura.

Ejemplo 1:

```
SRL $t0,$t1,5
```

```
opcode = 00000
rs      = 00000
rt      = 01111 ($t1)
rd      = 01110 ($t0)
param   = 00101
func    = 000001
```

En esta instrucción **param** representa el desplazamiento inmediato.

Ejemplo 2:

```
SRLR $t0,$t1,$t2
```

```
opcode = 00000
rs      = 10000 ($t2)
rt      = 01111 ($t1)
rd      = 01110 ($t0)
param   = 00000
func    = 000100
```

En esta instrucción el campo **param** debe codificarse con el valor cero y no participa en la operación. La cantidad de bits a desplazar se obtiene de los cinco bits menos significativos de R[rs].

6.3. Formato I-Type

El formato I (*immediate*)² dispone de los campos **opcode**, **rs**, **rt** e **imm** como se muestra en la figura 6.2. En este formato el campo **opcode** identifica la instrucción.

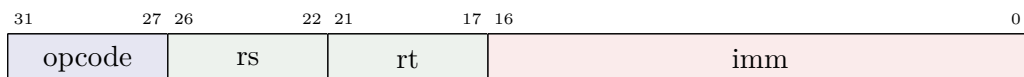


Figura 6.2: Formato tipo I

En la mayoría de las instrucciones los campos **rs** y **rt** codifican registros de propósito general, mientras que **imm** contiene una constante inmediata de 17 bits cuyo significado depende de la instrucción. Según la operación puede representar:

- un desplazamiento para direccionamiento indexado;
- un desplazamiento relativo para instrucciones de salto condicional;
- un desplazamiento relativo a un registro para instrucciones de enlazado indirecto;

²El nombre proviene del hecho de que la instrucción incorpora un operando inmediato codificado dentro de la propia instrucción.

- una constante inmediata para operaciones aritméticas o lógicas.

Algunas familias de instrucciones reinterpretan parte del campo `rt` para extender el código de operación o ampliar el tamaño del operando inmediato. Esta técnica permite conservar el ancho fijo de 32 bits sin introducir nuevos formatos de instrucción. Los casos particulares se describen en las secciones correspondientes.

6.3.1. Direccionamiento indexado

El direccionamiento indexado es el mecanismo utilizado por las instrucciones de acceso a memoria para obtener la dirección del operando. En este modo la instrucción no codifica una dirección absoluta, sino un registro base `R[rs]` y un desplazamiento inmediato `imm` a partir de los cuales la CPU calcula la dirección efectiva mediante la función `EA()`, definida en la Sección 4.6.1.

Todas las instrucciones de acceso a memoria del formato I emplean este mecanismo antes de realizar la lectura o escritura correspondiente, incluyendo accesos de palabra, media palabra y byte.

Ejemplo 1:

```
LW $t0, 8($sp)

opcode = 01000
rs      = 11110 ($sp)
rt      = 01110 ($t0)
imm     = 0 0000 0000 0000 1000 (8)
```

Dado `R[$sp] = 0x1000`, aplicando la ecuación (4.7) tenemos que `EA = 0x1008` por lo que `R[$t0] = M[0x1008]`. El contenido de la posición de memoria `0x1008` se copia en el registro `$t0`.

Ejemplo 2:

```
SW $t1, -4($sp)

opcode = 01001
rs      = 11110 ($sp)
rt      = 01111 ($t1)
imm     = 1 1111 1111 1111 1100 (-4)
```

Aplicando un razonamiento similar `EA = 0x0FFC` por lo que `M[0x0FFC] = R[$t1]`. El contenido del registro `$t1` se copia en la posición de memoria `0x0FFC`.

ATENCIÓN

La función `EA()` calcula únicamente la dirección efectiva. Corresponde a cada instrucción verificar que dicha dirección satisfaga los requisitos de alineación establecidos por la arquitectura. Las instrucciones que operan sobre palabras requieren direcciones múltiplo de cuatro, mientras que las que operan sobre medias palabras requieren direcciones múltiplo de dos. Cuando una instrucción que exige alineación utiliza una dirección no válida, la CPU genera la excepción correspondiente.

Como el desplazamiento se representa en complemento a dos, es posible acceder a posiciones ubicadas antes o después de la dirección base sin modificar previamente el contenido

del registro.

El formato I también se utiliza para las instrucciones de salto condicional. En ese caso, el operando inmediato deja de representar un desplazamiento sobre una dirección base y pasa a expresar un desplazamiento relativo al contador de programa.

6.3.2. Saltos condicionales

Las instrucciones de salto condicional (*branch*) modifican el flujo normal de ejecución únicamente cuando se cumple una condición lógica. En caso contrario, la ejecución continúa con la siguiente instrucción. De ser verdadera entonces la CPU realiza el salto cambiando el valor del PC a uno nuevo definido por $RA()$ según la sección 4.6.3. Como `imm` está representado en complemento a dos con 17 bits, el alcance del salto es aproximadamente $\pm 2^{16}$ instrucciones, equivalente a $\pm 2^{18}$ bytes (≈ 256 Ki instrucciones o ≈ 1 MiB de código).

Ejemplo 1:

```
BEQ $t0, $t1, 6

opcode = 10000
rs      = 01110 ($t0)
rt      = 01111 ($t1)
imm     = 0000 0000 0000 0110 (6)
```

Supongamos que la instrucción se encuentra en la dirección $PC = 0x00001000$. Si $R[\$t0] == R[\$t1]$, aplicando la ecuación correspondiente se obtiene:

$$RA = 0x1000 + 4 + (6 \times 4) = 0x101C$$

La ejecución continúa entonces en la dirección $0x101C$. En caso contrario, la condición resulta falsa y la CPU continúa normalmente con la siguiente instrucción ubicada en $0x1004$.

Ejemplo 2:

```
BNE $t0, $zero, -3

opcode = 10001
rs      = 01110 ($t0)
rt      = 00000 ($zero)
imm     = 1111 1111 1111 1101 (-3)
```

Supongamos nuevamente que la instrucción se encuentra en $PC = 0x00001000$. Si $R[\$t0] != 0$, el desplazamiento negativo permite regresar a una instrucción anterior:

$$RA = 0x1000 + 4 + (-3 \times 4) = 0xFF8$$

Este tipo de desplazamiento es muy utilizado para implementar bucles (*loops*), ya que permite volver a ejecutar un bloque de instrucciones sin utilizar direcciones absolutas.

6.3.3. Operaciones inmediatas

Las instrucciones de operación inmediata utilizan el campo `imm` como un operando constante incorporado en la propia instrucción, evitando la necesidad de cargar previamente dicho valor en un registro.

En RTM32 este grupo está formado por las instrucciones **ADDI**, **SLTI** y **SLTIU**. En todos los casos el operando inmediato ocupa 17 bits y se interpreta como un número en complemento a dos. Antes de utilizarse es extendido al tamaño de palabra mediante la función $E_{17}(imm, S)$ definida en la sección 4.4.1. De este modo pueden representarse constantes positivas y negativas dentro del rango permitido por el campo inmediato.

La instrucción **ADDI** realiza la suma del contenido de un registro con una constante inmediata.

Ejemplo:

```
ADDI $t0, $t1, -4

opcode = 00100
rs      = 01111 ($t1)
rt      = 01110 ($t0)
h       = 0
imm     = 1111 1111 1111 1100 (-4)
```

El operando inmediato se extiende con signo mediante $E_{17}(simm, S)$ y posteriormente se suma al contenido del registro **\$t1**. El resultado se almacena en **\$t0**.

Las instrucciones **SLTI** y **SLTIU** realizan una comparación entre el contenido de un registro y una constante inmediata. Si la condición resulta verdadera almacenan el valor 1 en el registro destino; en caso contrario almacenan 0. Todos los operadores relacionales realizan comparaciones con signo, salvo cuando alguno de sus operandos aparece envuelto por la función $U()$, que fuerza una interpretación sin signo.

Ejemplo:

```
SLTI $t0, $a0, 10

opcode = 10110
rs      = 01000 ($a0)
rt      = 01110 ($t0)
imm     = 00000000000001010 (10)
```

El operando inmediato se extiende con signo hasta el tamaño de palabra y se compara con el contenido del registro **\$a0**. Si **\$a0** es menor que 10, la instrucción almacena el valor 1 en **\$t0**; en caso contrario almacena 0.

6.3.4. Operaciones inmediatas extendidas

Algunas instrucciones de operación inmediata no requieren un desplazamiento de 17 bits. En consecuencia, el bit más significativo del operando inmediato se reutiliza como extensión del código de operación, reduciendo el tamaño de la constante a 16 bits y duplicando el número de instrucciones disponibles dentro de esta familia.

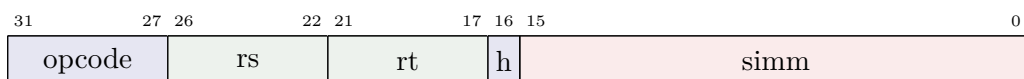


Figura 6.3: Reinterpretación del formato I para las operaciones inmediatas extendidas

Las instrucciones de esta familia comparten el mismo formato básico de las operaciones inmediatas convencionales. Sin embargo, el bit más significativo del campo **imm**, denominado

h, deja de formar parte del operando inmediato y pasa a integrarse al código de operación. El operando resultante, denominado **simm**, queda constituido únicamente por los 16 bits restantes.

Cada valor del campo **opcode** identifica una familia de dos instrucciones. El bit **h** selecciona la operación concreta, duplicando el espacio de codificación disponible sin modificar el formato de la instrucción.

Las instrucciones **ANDI**, **ANI**, **ORI** y **XORI** utilizan **simm** como una constante inmediata que, es extendida mediante $E_{16}(\text{simm}, z)$ definida en la sección 4.4.1 donde el parámetro z indica el tipo de extensión requerido por la instrucción (signo, cero o uno).

Por otra parte, las instrucciones **LCI**, **ANH**, **ORH** y **XORH** interpretan **simm** como la mitad más significativa de una constante de 32 bits. En estos casos la constante completa se obtiene mediante la función $C_{16}(\text{simm}, z)$ definida en la sección 4.4.2 donde el parámetro z indica el registro a concatenar requerido por la instrucción (**R[rs]** o **\$0**), permitiendo construir máscaras y constantes de 32 bits mediante una única instrucción.

Ejemplo 1:

```
ANDI $t0, $t1, 0xFF

opcode = 00100
rs      = 01111 ($t1)
rt      = 01110 ($t0)
h       = 0
imm     = 0000 0000 1111 1100 (-4)
```

El operando inmediato se extiende con ceros mediante $E_{16}(\text{simm}, 0)$ y posteriormente se realiza la operación AND bit a bit con el contenido del registro **\$t1**. El resultado se almacena en **\$t0**. Si **\$t1** contiene el valor **0x45003211** el resultado de **\$t0** = **0x00000011**.

Ejemplo 2:

```
LCI $t0, $t0, 0x1234

opcode = 00100
rs      = 01110 ($t0)
rt      = 01110 ($t0)
h       = 1
imm     = 0001 0010 0011 0100
```

La constante inmediata se concatena con los 16 bits menos significativos de **\$t0**, obteniéndose:

$$R[\$t0] = C_{16}(0x1234, R[\$t0])$$

Como resultado, los 16 bits más significativos del registro pasan a contener el valor **0x1234**, mientras que los 16 bits menos significativos conservan su contenido original.

Este mecanismo constituye otro ejemplo de reutilización jerárquica de la codificación de instrucciones. Al reducir el tamaño del operando inmediato en un bit es posible duplicar el número de instrucciones disponibles dentro de la familia de operaciones inmediatas sin introducir un nuevo formato de instrucción.

6.3.5. Saltos indirectos

En la mayoría de las instrucciones de formato I, los campos **rs**, **rt** e **imm** representan dos registros y un operando inmediato. Sin embargo, algunas familias de instrucciones reutilizan parte de los bits del campo **rt** para extender el código de operación. En estas instrucciones el campo **rt** deja de codificar un registro de propósito general y pasa a formar parte del proceso de decodificación, permitiendo definir variantes de una misma operación sin introducir un nuevo formato de instrucción.

Las instrucciones **JR**, **JALR** y **JALRX** utilizan este mecanismo. Todas ellas comparten el mismo **opcode** y se diferencian mediante los bits almacenados en el campo **rt**. Al mismo tiempo, los bits restantes de dicho campo permiten seleccionar el registro de enlace en la variante **JALRX**.

La figura 6.4 muestra la reinterpretación de los campos.

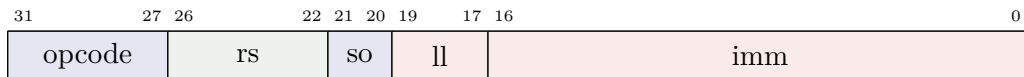


Figura 6.4: Reinterpretación del formato I para JR/JALR

Los dos bits más significativos del campo **rt** constituyen un subcódigo de operación (**so**) que identifica la variante de la instrucción, mientras que los tres bits menos significativos (**ll**) se utilizan para extender la constante **imm** formando la constante inmediata **llimm** de 20 bits de longitud.

Todas las instrucciones de esta familia realizan un salto hacia una dirección calculada a partir de un registro base y un desplazamiento inmediato. A diferencia de los saltos condicionales, donde el desplazamiento es relativo al contador de programa, en este caso el desplazamiento es relativo al contenido del registro **rs** y la dirección de destino se obtiene mediante la función $AA_{22}(R[rs], llimm)$ definida en la sección 4.6.4.

La instrucción **JR** (**so** = b00) realiza el salto sin conservar la dirección de retorno. Por el contrario, **JALR** (**so** = b01) almacena previamente el valor $PC + 4$ en el registro de enlace **\$ra**. Finalmente, **JALRX** generaliza este comportamiento permitiendo seleccionar uno de los cuatro registros de enlace (**\$lr0..\$lr3**) mediante dos bits específicos (**lr**) dentro del campo **rt** codificados como se muestra en la figura 6.5.

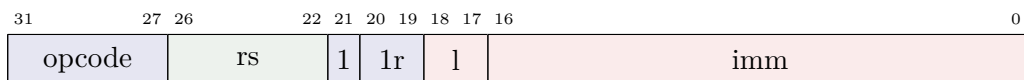


Figura 6.5: Reinterpretación del formato I para JALRX

Notar que ahora **imm** se extiende a 19 bits **limm** y la dirección de destino se obtiene mediante $AA_{21}(R[rs], limm)$ que solo difiere en el tamaño de la constante inmediata que es menor. Esta capacidad resulta útil cuando existen múltiples contextos de llamada o cuando el software necesita mantener varios enlaces activos simultáneamente.

Ejemplo 1:

```
JR $t0, 8

opcode = 00010
rs      = 01110 ($t0)
rt      = 00 000
imm     = 0 0000 0000 0000 1000
```

```
l1imm = 000 0 0000 0000 0000 1000 (8)
```

Si $R[\$t0] = 0x2000$, aplicando la ecuación (4.10) se obtiene:

$$AA = 0x2000 + (8 \times 4) = 0x2020$$

La CPU transfiere el control a la dirección $0x2020$ sin modificar ningún registro de enlace.

Ejemplo 2:

```
JALR $s0, -2

opcode = 00010
rs      = 10000 ($s0)
rt      = 01 111
imm     = 1 1111 1111 1111 1110

l1imm   = 111 1 1111 1111 1111 1110 (-2)
```

Si $R[\$s0] = 0x4000$ y la instrucción se encuentra en $PC = 0x1000$, entonces:

$$AA = 0x4000 + (-2 \times 4) = 0x3FF8$$

Antes de efectuar el salto se almacena $0x1004$ en el registro $\$ra$. Posteriormente la ejecución continúa en la dirección $0x3FF8$.

Ejemplo 3:

```
JALRX $t1, 12

opcode = 00010
rs      = 01111 ($t1)
rt      = 1 01 00
imm     = 0 0000 0000 0000 1100

l1imm   = 00 0 0000 0000 0000 1100 (12)
```

En este ejemplo el bit más significativo de **rt**, con valor 1 identifica la instrucción **JALRX**, los dos bits que le siguen (01) seleccionan el registro de enlace correspondiente y los restantes forman parte de la constante **s1imm**. El valor $PC + 4$ se almacena en $\$lr1$ y la ejecución continúa en la dirección calculada mediante la ecuación (4.10).

Este mecanismo constituye un ejemplo de reutilización jerárquica de la codificación de instrucciones. Un mismo **opcode** identifica una familia de operaciones relacionadas, mientras que parte de un campo originalmente destinado a operandos se reutiliza como extensión del código de operación y otra parte para extender constantes. De este modo la arquitectura incrementa el número de instrucciones disponibles sin aumentar el tamaño de la palabra de instrucción ni introducir nuevos formatos de codificación.

6.4. Formato J-Type

El formato J (*jump*) está destinado a las instrucciones de transferencia incondicional de control mediante direccionamiento relativo al contador de programa (PC). Dispone únicamente de los campos **opcode** y **jump**, como se muestra en la figura 6.6. En este formato el campo

opcode identifica la familia de instrucciones, mientras que el resto de la palabra se destina a codificar el desplazamiento del salto.

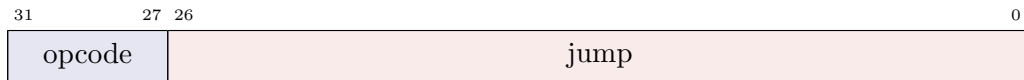


Figura 6.6: Formato tipo J

En su forma básica el campo **jump** representa un desplazamiento relativo al contador de programa. Sin embargo, al igual que ocurre en otras familias de instrucciones de RTM32, algunas instrucciones reinterpretan una parte de este campo para extender el código de operación o seleccionar operandos adicionales. Esta técnica permite incrementar el número de instrucciones disponibles manteniendo un tamaño fijo de 32 bits.

6.4.1. Saltos relativos

A diferencia de las instrucciones de salto indirecto, donde el destino se calcula a partir de un registro base, estas instrucciones de salto incondicional utilizan un desplazamiento relativo al contador de programa (PC). La dirección de salto depende únicamente del valor actual del PC y del operando inmediato.

Las instrucciones J y JAL reutilizan los dos bits más significativos del campo **jump** para extender el código de operación, mientras que los veinticinco bits restantes forman un operando inmediato denominado **vlimm** que se utiliza para obtener la dirección de destino $RA_{27}(elimm)$ usando la ecuación 4.9.



Figura 6.7: Reinterpretación del formato J para J/JAL

La reinterpretación del formato se muestra en la figura 6.7. El subcódigo de operación (**so**) distingue ambas variantes:

- **so** = b00: instrucción J.
- **so** = b01: instrucción JAL.

La instrucción J transfiere el control a la dirección calculada sin almacenar una dirección de retorno. Por el contrario, JAL guarda previamente el valor PC+4 en el registro **\$ra** antes de efectuar el salto.

La instrucción JALX constituye una extensión de esta familia. Para incorporar la selección del registro de enlace consume un bit adicional del operando inmediato, reduciendo su longitud efectiva. La reinterpretación del formato se muestra en la figura 6.8.



Figura 6.8: Reinterpretación del formato J para JALX

En este caso el bit más significativo del campo **jump** identifica la subfamilia JALX, mientras que los dos bits siguientes seleccionan uno de los cuatro registros de enlace (**\$lr0..\$lr3**). Los veinticuatro bits restantes forman la constante inmediata **limm** utilizados para obtener

la dirección de destino mediante $RA_{26}(vlimm)$. La reducción de un bit en el tamaño del operando inmediato permite disponer de cuatro registros de enlace sin modificar el formato base de la instrucción.

Ejemplo 1:

[illegible]

Suponiendo que la instrucción se encuentra en PC = 0x00001000, se obtiene:

$$RA = 0x1000 + 4 + (32 \times 4) = 0x1084$$

La ejecución continúa en la dirección 0x1084.

Ejemplo 2:

```
JAL -8

opcode = 00001
jump   = 01 111111111111111111111111000

so      = 01
elimm   = 1 1111 1111 1111 1111 1111 1000 (-8)
```

Si la instrucción se encuentra en $PC = 0x00001000$, la CPU almacena previamente $0x1004$ en el registro $\$ra$ y transfiere el control a la dirección calculada mediante la ecuación (4.9).

Ejemplo 3:

```
JALX $lr2, 12

opcode = 00001
jump   = 1 10 0000000000000000000000001100

lr      = 10
vlimm   = 0000 0000 0000 0000 0000 1100 (12)
```

En este ejemplo el bit más significativo identifica la instrucción JALX, los dos bits siguientes seleccionan el registro de enlace \$lr2 y los restantes forman parte de la constante inmediata limm. Antes del salto se almacena PC+4 en el registro de enlace seleccionado y la ejecución continúa en la dirección calculada mediante la ecuación (4.9).

7. Instrucciones del Procesador STX4

7.1. Instrucciones aritméticas

Las instrucciones aritméticas realizan operaciones enteras sobre el contenido de los registros de propósito general. Dependiendo de la instrucción, los operandos pueden provenir de dos registros o de un registro y un valor inmediato codificado dentro de la propia instrucción.

Esta familia incluye instrucciones de suma `ADD` `ADDI`, resta `SUB`, multiplicación `MUL` `MULH` `MULHU` `MULHSU`, división `DIV` `DIVU` y cálculo del resto `REST` `RESTU`. Según la operación, las instrucciones pertenecen a los formatos `R` o `I`.

Cuando una instrucción emplea un operando inmediato, éste se interpreta como un entero con signo de 17 bits y se extiende al tamaño nativo del procesador mediante el operador $E_{17}(\text{imm}, S)$ definido previamente.

Todas las instrucciones de esta familia actualizan las banderas de estado `Z`, `N`, `O` y `C`. Estas indican, respectivamente, si el resultado es nulo, negativo, produjo desbordamiento aritmético o generó un acarreo (en una suma) o un préstamo (en una resta). En aquellas operaciones donde una bandera no resulte aplicable, su valor queda indefinido salvo que se especifique expresamente lo contrario. El desbordamiento aritmético no genera una excepción y únicamente modifica el estado de la bandera correspondiente.

7.1.1. Suma y resta

Las instrucciones de suma y resta constituyen las operaciones aritméticas básicas de la arquitectura. Permiten realizar operaciones entre registros de propósito general o entre un registro y un operando inmediato, almacenando el resultado en un registro destino.

Esta subfamilia está formada por las instrucciones `ADD`, `ADDI` y `SUB`. Las instrucciones `ADD` y `SUB` pertenecen al formato `R`, mientras que `ADDI` utiliza el formato `I`.

Instrucción	Operación
<code>ADD</code>	$R[rd] = R[rs] + R[rt]$
<code>ADDI</code>	$R[rt] = R[rs] + E_{17}(\text{imm}, S)$
<code>SUB</code>	$R[rd] = R[rs] - R[rt]$

En las instrucciones `ADD` y `SUB`, ambos operandos provienen de registros de propósito general. En `ADDI`, el segundo operando corresponde a un valor inmediato con signo de 17 bits, extendido al tamaño nativo del procesador mediante el operador $E_{17}(\text{imm}, S)$.

Estas instrucciones se utilizan para implementar cálculos enteros, actualización de contadores, manipulación de direcciones, desplazamientos mediante constantes y, en general, cualquier expresión aritmética basada en sumas y restas.

Sintaxis y ejemplos

```
add  $t3, $t1, $t2
addi $t4, $t4, -8
sub  $t5, $t3, $t2
```

Suponiendo inicialmente:

$$t1 = 25, \quad t2 = 17, \quad t4 = 100$$

la ejecución produce:

$$t3 = 42, \quad t4 = 92, \quad t5 = 25$$

Al finalizar cada instrucción se actualizan las banderas de estado **Z**, **N**, **O** y **C**, según las reglas descritas en la Sección 7.1.1.

Actualización de banderas

Las instrucciones **ADD**, **SUB** y **ADDI** actualizan las banderas **Z**, **N**, **O** y **C** de acuerdo con el resultado de la operación aritmética.

La bandera **Z** se activa cuando el resultado obtenido es cero, mientras que la bandera **N** refleja el bit más significativo del resultado, indicando una interpretación negativa en complemento a dos.

Las banderas **C** y **O** representan condiciones diferentes. La bandera **C** indica un acarreo fuera del tamaño del operando en una suma o un préstamo en una resta, por lo que está asociada a la interpretación sin signo de los valores. En cambio, la bandera **O** indica que el resultado no puede representarse correctamente utilizando complemento a dos, por lo que está asociada a la interpretación con signo.

La siguiente suma de dos operandos de 32 bits:

$$0xFFFFFFFF + 1 = 0x00000000$$

produce:

$$Z = 1, \quad N = 0, \quad C = 1, \quad O = 0$$

El acarreo indica que la suma excedió el rango de representación sin signo, pero no existe desbordamiento con signo porque el resultado equivalente es cero $(-1 + 1)$.

En cambio:

$$0x7FFFFFFFF + 1 = 0x80000000$$

produce:

$$Z = 0, \quad N = 1, \quad C = 0, \quad O = 1$$

En este caso el resultado no puede representarse como un entero positivo con signo, por lo que se activa la bandera de desbordamiento.

En las operaciones de resta, la bandera **C** indica que la operación generó un préstamo (*borrow*) fuera del bit más significativo del operando. Se activa cuando el minuendo es menor que el sustraendo bajo una interpretación sin signo.

Por ejemplo:

$$3 - 5 = 0xFFFFFFFF$$

produce:

$$Z = 0, \quad N = 1, \quad C = 1, \quad O = 0$$

La operación requiere un préstamo, por lo que **C** se activa. El resultado almacenado corresponde a la representación en complemento a dos de -2 , pero la bandera **C** refleja únicamente la condición de préstamo de la operación sin signo.

NOTA

La bandera **C** permite detectar condiciones de acarreo y préstamo para aritmética sin signo, mientras que la bandera **O** permite detectar desbordamientos en aritmética con signo. Ambas son independientes y pueden tomar cualquier combinación de valores según la operación realizada.

La separación entre las banderas **C** y **O** permite implementar decisiones aritméticas sobre datos con y sin signo utilizando la misma operación. Por ejemplo, para determinar si un valor sin signo contenido en **\$1** es menor que otro contenido en **\$2**, basta ejecutar la resta **\$1 - \$2** y consultar posteriormente la bandera **C**. De esta forma, una única operación aritmética proporciona simultáneamente el resultado y la información necesaria para evaluar la relación entre ambos operandos.

7.1.2. Multiplicación

Las instrucciones de multiplicación realizan el producto entre dos registros de propósito general y almacenan en el registro destino una porción del resultado de 64 bits generado por la unidad aritmética. Dependiendo de la instrucción, los operandos pueden interpretarse como enteros con signo o sin signo, y el valor almacenado corresponde a la mitad inferior o superior del producto.

Esta subfamilia está formada por las instrucciones **MUL**, **MULH**, **MULHU** y **MULHSU**. Todas pertenecen al formato **R**.

Instrucción	Operación
MUL	$R[rd] = \{R[rs] \times R[rt]\}_{[31:0]}$
MULH	$R[rd] = \{R[rs] \times R[rt]\}_{[63:32]}$
MULHU	$R[rd] = \{U(R[rs]) \times U(R[rt])\}_{[63:32]}$
MULHSU	$R[rd] = \{R[rs] \times U(R[rt])\}_{[63:32]}$

Internamente todas las instrucciones realizan una multiplicación completa de 64 bits. La diferencia entre ellas radica únicamente en la interpretación de los operandos y en la mitad del producto que finalmente se escribe en el registro destino.

La instrucción **MUL** devuelve los 32 bits menos significativos del producto. Las restantes instrucciones permiten obtener los 32 bits más significativos bajo distintas interpretaciones de signo, facilitando la implementación de algoritmos de precisión extendida, aritmética multiprecisión y operaciones sobre enteros sin signo.

Sintaxis y ejemplos

```
mul    $t2, $t0, $t1
mulh   $t3, $t0, $t1
mulhu  $t4, $t0, $t1
mulhsu $t5, $t0, $t1
```

Si inicialmente:

$$t0 = 300000, \quad t1 = 200000$$

el producto completo es:

$$300000 \times 200000 = 60000000000$$

La instrucción **MUL** almacena en el registro destino los 32 bits menos significativos del producto, mientras que **MULH** almacena los 32 bits más significativos del mismo resultado.

Las instrucciones **MULHU** y **MULHSU** producen un resultado diferente únicamente cuando alguno de los operandos representa un valor negativo en complemento a dos. En ese caso, la interpretación con signo o sin signo modifica el producto de 64 bits y, por consiguiente, la mitad superior obtenida.

Por ejemplo, considerando:

$$R[rs] = -1$$

y

$$R[rt] = 2$$

la instrucción **MULH** interpreta ambos operandos como enteros con signo, mientras que **MULHU** considera ambos operandos sin signo y **MULHSU** interpreta únicamente el primer operando como un entero con signo.

Esta diferencia resulta fundamental en algoritmos de aritmética multiprecisión y en implementaciones eficientes de compiladores para lenguajes que distinguen entre tipos enteros con y sin signo.

Actualización de banderas

Las instrucciones de multiplicación actualizan las banderas de estado **Z**, **N**, **O** y **C**.

La bandera **Z** se activa cuando el valor finalmente almacenado en el registro destino es igual a cero.

La bandera **N** refleja el bit más significativo del valor escrito en el registro destino. En consecuencia, para las instrucciones que devuelven la mitad superior del producto, la bandera corresponde al bit más significativo de dicha mitad y no del producto completo.

La bandera **O** indica que el producto completo de 64 bits no puede representarse utilizando un entero con signo de 32 bits. Equivalente a ello, **O** se activa cuando la mitad superior del producto no corresponde a la extensión de signo de la mitad inferior.

En otras palabras, si los bits comprendidos entre las posiciones 63 y 32 no son todos iguales al bit 31 del resultado, el producto excede el rango representable por un entero con

signo de 32 bits y se produce desbordamiento aritmético.

Por ejemplo:

$$50000 \times 50000 = 2500000000$$

El resultado requiere más de 31 bits para representar correctamente el valor con signo, por lo que la bandera **O** se activa aun cuando la instrucción **MUL** únicamente almacene la mitad inferior del producto.

La bandera **C** tiene un significado distinto según la instrucción ejecutada.

Para la instrucción **MUL**, la bandera **C** indica que existen bits significativos en la mitad superior del producto de 64 bits que no fueron almacenados en el registro destino. En otras palabras, informa que el resultado completo no cabe en los 32 bits devueltos por la instrucción.

Por ejemplo:

$$0x00010000 \times 0x00010000 = 0x00000001\ 00000000$$

La instrucción **MUL** almacena:

$$0x00000000$$

mientras que la mitad superior contiene:

$$0x00000001$$

En consecuencia:

$$Z = 1, \quad C = 1$$

ya que el registro destino contiene cero, aunque el producto completo no sea nulo.

En las instrucciones **MULH**, **MULHU** y **MULHSU** la bandera **C** no tiene significado, puesto que precisamente se devuelve la mitad superior del producto. En consecuencia, su valor queda indefinido.

NOTA

La combinación de las instrucciones **MUL** y **MULH** permite obtener el producto completo de 64 bits entre dos enteros con signo. De forma análoga, las instrucciones **MULHU** y **MULHSU** permiten recuperar la mitad superior del producto cuando los operandos deben interpretarse como valores sin signo o de signo mixto.

7.1.3. División

Las instrucciones de división realizan el cociente entero entre dos registros de propósito general y almacenan el resultado en un tercer registro. Dependiendo de la instrucción, los operandos se interpretan como enteros con signo o sin signo.

Esta subfamilia está formada por las instrucciones **DIV** y **DIVU**. Ambas pertenecen al formato **R**.

Instrucción	Operación
DIV	$R[rd] = R[rs] \div R[rt]$ (con signo)
DIVU	$R[rd] = U(R[rs]) \div U(R[rt])$ (sin signo)

La instrucción DIV interpreta ambos operandos como enteros con signo representados en complemento a dos, mientras que DIVU los interpreta como enteros sin signo.

En ambos casos el cociente corresponde a una división entera, por lo que cualquier parte fraccionaria es descartada.

Sintaxis y ejemplos

```
div  $t2, $t0, $t1
divu $t3, $t0, $t1
```

Si inicialmente:

$$t0 = 100, \quad t1 = 8$$

la ejecución produce:

$$t2 = 12, \quad t3 = 12$$

ya que ambas interpretaciones producen el mismo resultado para operandos positivos.

En cambio, considerando:

$$R[rs] = -20, \quad R[rt] = 3$$

la instrucción DIV devuelve el cociente correspondiente a la división con signo, mientras que DIVU interpreta ambos registros como enteros sin signo, obteniendo un resultado completamente diferente.

Condiciones de excepción

La división requiere que el divisor sea distinto de cero.

Si el contenido del registro `rt` es igual a cero, las instrucciones DIV y DIVU generan una excepción por instrucción ilegal y no modifican el registro destino.

En la instrucción DIV existe además un caso especial de desbordamiento aritmético cuando el dividendo corresponde al menor entero representable en 32 bits con signo y el divisor es -1 .

$$-2147483648 \div (-1)$$

Este resultado no puede representarse mediante un entero de 32 bits en complemento a dos. En consecuencia, la arquitectura genera una excepción por instrucción ilegal y la operación no produce ningún resultado.

La instrucción DIVU no presenta esta condición de desbordamiento, ya que todos los cocientes posibles entre operandos sin signo de 32 bits son representables utilizando el mismo tamaño de palabra.

NOTA

A diferencia de las operaciones de suma, resta y multiplicación, las condiciones de error durante una división no se reflejan mediante las banderas de estado. Cuando ocurre una división por cero o un desbordamiento en la instrucción **DIV**, la operación genera inmediatamente una excepción y no produce un resultado válido.

Actualización de banderas

Cuando la división finaliza correctamente, las instrucciones **DIV** y **DIVU** actualizan las banderas de estado **Z**, **N**, **O** y **C**.

La bandera **Z** se activa cuando el cociente almacenado en el registro destino es igual a cero.

La bandera **N** refleja el bit más significativo del cociente almacenado.

Las banderas **O** y **C** no tienen significado para las operaciones de división y, en consecuencia, su valor queda indefinido tras la ejecución de estas instrucciones.

Por ejemplo:

$$5 \div 8 = 0$$

produce:

$$Z = 1, \quad N = 0$$

ya que el cociente entero es cero.

Considerando ahora:

$$-20 \div 3 = -6$$

la instrucción **DIV** produce un resultado negativo, por lo que la bandera **N** se activa.

Las condiciones de división por cero y desbordamiento descritas anteriormente impiden la finalización normal de la instrucción. En esos casos no se actualiza ninguna bandera de estado debido a que la ejecución se interrumpe mediante una excepción.

NOTA

Las instrucciones de división únicamente devuelven el cociente. Para obtener el resto de una división deben utilizarse las instrucciones de la subfamilia **REST** y **RESTU**, descritas en la siguiente sección.

7.1.4. Resto

Las instrucciones de cálculo del resto obtienen el residuo de una división entera entre dos registros de propósito general y almacenan el resultado en un tercer registro. Dependiendo de la instrucción, los operandos se interpretan como enteros con signo o sin signo.

Esta subfamilia está formada por las instrucciones **REST** y **RESTU**. Ambas pertenecen al formato **R**.

Instrucción	Operación
REST	$R[rd] = R[rs] \text{ mód } R[rt] \text{ (con signo)}$
RESTU	$R[rd] = U(R[rs]) \text{ mód } U(R[rt]) \text{ (sin signo)}$

La instrucción **REST** interpreta ambos operandos como enteros con signo representados en complemento a dos, mientras que **RESTU** los interpreta como enteros sin signo.

En ambos casos el resultado corresponde al resto de la división entera entre ambos operandos. La siguiente relación se cumple siempre que la operación finaliza correctamente:

$$\text{dividendo} = (\text{cociente} \times \text{divisor}) + \text{resto}$$

donde el cociente es el producido por las instrucciones **DIV** o **DIVU**, según corresponda.

Sintaxis y ejemplos

```
rest  $t2, $t0, $t1
restu $t3, $t0, $t1
```

Si inicialmente:

$$t0 = 100, \quad t1 = 8$$

la ejecución produce:

$$t2 = 4, \quad t3 = 4$$

ya que ambas interpretaciones producen el mismo resultado para operandos positivos.

En cambio, considerando:

$$R[rs] = -20, \quad R[rt] = 3$$

la instrucción **REST** calcula el resto utilizando una división con signo, mientras que **RESTU** interpreta ambos registros como enteros sin signo, obteniendo un resultado diferente.

Condiciones de excepción

Las instrucciones de cálculo del resto requieren que el divisor sea distinto de cero.

Si el contenido del registro **rt** es igual a cero, las instrucciones **REST** y **RESTU** generan una excepción por instrucción ilegal y no modifican el registro destino.

Al igual que ocurre con la instrucción **DIV**, la instrucción **REST** presenta un caso especial de desbordamiento cuando el dividendo corresponde al menor entero representable en 32 bits con signo y el divisor es igual a -1 .

$$-2147483648 \text{ mód } (-1)$$

Aunque matemáticamente el resultado es cero, esta operación no se encuentra definida por la arquitectura debido a que requiere evaluar una división cuyo cociente no puede representarse mediante un entero con signo de 32 bits. En consecuencia, la arquitectura genera una excepción por instrucción ilegal y la operación no produce ningún resultado.

La instrucción **RESTU** no presenta esta condición de desbordamiento, ya que todos los resultados posibles entre operandos sin signo son representables utilizando el tamaño nativo de palabra.

NOTA

Las instrucciones **REST** y **DIV** comparten exactamente las mismas condiciones de excepción. Del mismo modo, **RESTU** y **DIVU** comparten el mismo comportamiento frente a errores de ejecución.

Actualización de banderas

Cuando la operación finaliza correctamente, las instrucciones **REST** y **RESTU** actualizan las banderas de estado **Z**, **N**, **O** y **C**.

La bandera **Z** se activa cuando el resto almacenado en el registro destino es igual a cero.

La bandera **N** refleja el bit más significativo del resultado almacenado.

Las banderas **O** y **C** no tienen significado para las operaciones de cálculo del resto y, en consecuencia, su valor queda indefinido tras la ejecución de estas instrucciones.

Por ejemplo:

$$20 \text{ mód } 5 = 0$$

produce:

$$Z = 1, \quad N = 0$$

ya que el resto de la división es cero.

Considerando ahora:

$$-20 \text{ mód } 3 = -2$$

la instrucción **REST** produce un resultado negativo en complemento a dos, por lo que la bandera **N** se activa.

Las condiciones de división por cero y desbordamiento descritas anteriormente interrumpen la ejecución normal de la instrucción mediante una excepción. En estos casos no se actualiza ninguna bandera de estado.

NOTA

Las instrucciones **DIV/REST** y **DIVU/RESTU** están diseñadas para utilizarse de forma complementaria. Ejecutadas sobre los mismos operandos permiten obtener, respectivamente, el cociente y el resto de una división entera.

7.2. Instrucciones lógicas

Las instrucciones lógicas realizan operaciones booleanas bit a bit sobre el contenido de los registros de propósito general. Estas operaciones permiten manipular máscaras, seleccionar

campos individuales dentro de una palabra, combinar valores y modificar grupos de bits mediante operaciones simples.

Las operaciones lógicas pueden utilizar dos operandos almacenados en registros o un registro junto con un operando inmediato codificado en la instrucción. Las instrucciones con operandos inmediatos utilizan diferentes formas de extensión según el tipo de operación, permitiendo trabajar tanto con máscaras de bits inferiores como con valores ubicados en la mitad superior del registro.

Las instrucciones lógicas actualizan las banderas de estado Z y N. La bandera Z se activa cuando el resultado obtenido es cero, mientras que N refleja el bit más significativo del resultado. Las banderas O y C no son modificadas por estas instrucciones.

La familia está formada por las instrucciones AND, OR, XOR, NOR, ANDI, ANI, ORI, XORI, ANH, ORH y XORH.

7.2.1. Operaciones lógicas entre registros

Las instrucciones AND, OR, XOR y NOR realizan operaciones lógicas bit a bit entre dos registros fuente y almacenan el resultado en un registro destino.

Todas estas instrucciones utilizan el formato R y se diferencian únicamente en la operación lógica aplicada por la ALU.

Instrucción	Operación
AND	$R[rd] = R[rs] \& R[rt]$
OR	$R[rd] = R[rs] \mid R[rt]$
XOR	$R[rd] = R[rs] \oplus R[rt]$
NOR	$R[rd] = \neg(R[rs] \mid R[rt])$

Estas instrucciones son utilizadas habitualmente para implementar máscaras, combinación de campos, inversión de bits y operaciones booleanas generales.

Sintaxis y ejemplos

```
and $t0, $t1, $t2
or  $t3, $t1, $t2
xor $t4, $t1, $t2
nor $t5, $t1, $t2
```

Si $t1 = 0xFF00FF00$ y $t2 = 0x0F0F0F0F$ entonces para la instrucción AND $t0 = 0x0F000F00$.

7.2.2. Diseño de las operaciones lógicas inmediatas

RTM32 distingue dos familias de operaciones lógicas inmediatas:

- ANDI, junto con ORI y XORI, utilizan la semántica clásica de las arquitecturas RISC, donde el operando inmediato representa una constante extendida al tamaño de palabra.
- ANI, junto con ORI, XORI, ANH, ORH y XORH, interpretan el operando inmediato como un modificador de media palabra, preservando automáticamente la mitad opuesta del registro.

Las instrucciones **ORI** y **XORI** pertenecen simultáneamente a ambas familias debido a que la extensión con ceros conserva naturalmente la mitad superior del registro al aplicar las operaciones **OR** y **XOR**. En cambio, la operación **AND** requiere dos instrucciones diferenciadas (**ANDI** y **ANI**) porque las extensiones con ceros y con unos producen semánticas distintas.

Justificación del diseño Las instrucciones **ORI** y **XORI** pertenecen simultáneamente a ambas familias. En estas operaciones la extensión del inmediato con ceros permite representar una constante de palabra completa y, al mismo tiempo, preservar automáticamente la mitad superior del registro debido a las propiedades del álgebra booleana:

$$x \vee 0 = x$$

$$x \oplus 0 = x$$

Por el contrario, la operación **AND** no posee esta propiedad. La extensión con ceros produce una máscara de palabra completa, mientras que la extensión con unos preserva automáticamente la mitad superior del registro:

$$x \wedge 0 = 0$$

$$x \wedge 1 = x$$

Por este motivo RTM32 define dos instrucciones distintas para la operación **AND** inmediata: **ANDI**, orientada al uso de máscaras completas siguiendo la semántica clásica de las arquitecturas RISC, y **ANI**, orientada a la modificación selectiva de la mitad inferior del registro preservando automáticamente la mitad superior.

7.2.3. Operaciones lógicas inmediatas

Las instrucciones **ANDI**, **ANI**, **ORI** y **XORI** realizan operaciones lógicas entre un registro y un valor inmediato de 16 bits. El valor inmediato es extendido al tamaño nativo del procesador antes de efectuar la operación. La extensión utilizada depende de la instrucción.

Instrucción	Operación
ANDI	$R[rt] = R[rs] \& E_{16}(imm, 0)$
ANI	$R[rt] = R[rs] \& E_{16}(imm, 1)$
ORI	$R[rt] = R[rs] E_{16}(imm, 0)$
XORI	$R[rt] = R[rs] \oplus E_{16}(imm, 0)$

La extensión utilizada por **ANI**, **ORI** y **XORI** garantiza que únicamente se modifiquen los 16 bits menos significativos del registro, preservando el contenido de la mitad superior.

Sintaxis y ejemplos

```
ani  $t0, $t1, 0xFF00
ori  $t2, $t3, 0x5678
xori $t4, $t5, 0xFFFF
```

Por ejemplo si $t3 = 0x12340000$ entonces la instrucción `ORI t2 = 0x12345678`.

La instrucción `ANDI` extiende el valor inmediato con ceros, sin preservar los valores de los bits más significativos de la operación. Permite crear máscaras de palabra completa. Esta semántica coincide con la utilizada por numerosas arquitecturas RISC modernas y facilita la generación eficiente de código por parte de los compiladores.

```
andi    $t0, $t1, 0xFF
```

En el ejemplo de la instrucción anterior la máscara `0xFF` se extiende al valor `0x000000FF`, permitiendo enmascarar los 8 bits menos significativos de `$t1`.

7.2.4. Operaciones lógicas inmediatas en la parte alta

Las instrucciones `ANH`, `ORH` y `XORH` constituyen la versión sobre la mitad superior de las operaciones lógicas inmediatas orientadas a campos. Permiten modificar únicamente los 16 bits más significativos del registro, preservando la mitad inferior. La operación se realiza utilizando una constante de 32 bits cuyo valor inmediato ocupa la mitad superior de la palabra. La mitad inferior se completa automáticamente con unos o ceros según la operación lógica aplicada.

Instrucción	Operación
<code>ANH</code>	$R[rt] = R[rs] \& H_{16}(imm, 1)$
<code>ORH</code>	$R[rt] = R[rs] H_{16}(imm, 0)$
<code>XORH</code>	$R[rt] = R[rs] \oplus H_{16}(imm, 0)$

La concatenación utilizada por estas instrucciones garantiza que únicamente se modifiquen los 16 bits más significativos del registro, preservando el contenido de la mitad inferior.

Estas instrucciones permiten modificar directamente la mitad superior de un registro sin utilizar desplazamientos adicionales.

7.3. Construcción de constantes

La instrucción `LCI` permite construir valores de 32 bits mediante la combinación de un operando inmediato de 16 bits con la mitad inferior de un registro fuente. Esta operación resulta especialmente útil para generar constantes, direcciones y valores que no pueden representarse mediante un único valor inmediato de 16 bits. A diferencia del resto de las instrucciones con operandos inmediatos, `LCI` no realiza una extensión del valor inmediato. En su lugar, utiliza el operador de concatenación $C_{16}()$ definido previamente para formar el resultado. La instrucción `LCI` no modifica las banderas de estado.

7.3.1. LCI

La operación realizada por la instrucción está definida por:

$$R[rt] = C_{16}(imm, R[rs])$$

Los 16 bits del operando inmediato constituyen la mitad más significativa del resultado, mientras que los 16 bits menos significativos se obtienen del registro fuente.

Sintaxis y ejemplos

```
lci $t0, $zero, 0x1234
```

Dado que zero está cableado a 0 la ejecución produce que $t0 = 0x12340000$.

La instrucción LCI puede combinarse con instrucciones lógicas inmediatas para construir constantes completas de 32 bits. Por ejemplo el siguiente código carga en $t0 = 0x12345678$.

```
lci $t0, $zero, 0x1234
ori $t0, $t0, 0x5678
```

También es posible construir la misma constante en el orden inverso obteniéndose nuevamente en $t0 = 0x12345678$

```
ori $t0, $zero, 0x5678
lci $t0, $t0, 0x1234
```

Esta propiedad proporciona flexibilidad para la generación y reorganización de secuencias de instrucciones que construyen constantes de 32 bits.

7.4. Instrucciones de desplazamiento

Las instrucciones de desplazamiento permiten desplazar o rotar los bits contenidos en un registro de propósito general. Dependiendo de la instrucción, la cantidad de posiciones a desplazar puede especificarse mediante un parámetro inmediato de 5 bits o tomarse de los cinco bits menos significativos de un registro.

Esta familia está formada por ocho instrucciones. Las instrucciones SLL, SRL, SRA y RLL utilizan un desplazamiento inmediato, mientras que SLLR, SRLR, SRAR y RLLR obtienen el desplazamiento desde un registro.

En las variantes controladas por registro únicamente se consideran los cinco bits menos significativos del registro fuente (**rs**), por lo que el desplazamiento efectivo pertenece al intervalo comprendido entre 0 y 31 posiciones.

Las instrucciones de esta familia no modifican las banderas de estado.

7.4.1. Desplazamientos lógicos

Las instrucciones SLL y SRL realizan desplazamientos lógicos hacia la izquierda y hacia la derecha, respectivamente. En ambos casos, las posiciones vacantes son completadas con ceros y los bits desplazados fuera del registro son descartados.

Instrucción	Operación
SLL	$R[rd] = R[rt] \ll param$
SRL	$R[rd] = R[rt] \gg param$

Matemáticamente, realizar un desplazamiento lógico a la izquierda de un valor X por n posiciones es estrictamente equivalente a una multiplicación entera por una potencia de dos, truncada a la capacidad del registro:

$$Y = X \cdot 2^n \quad (\text{mód } 2^{32}) \quad (7.1)$$

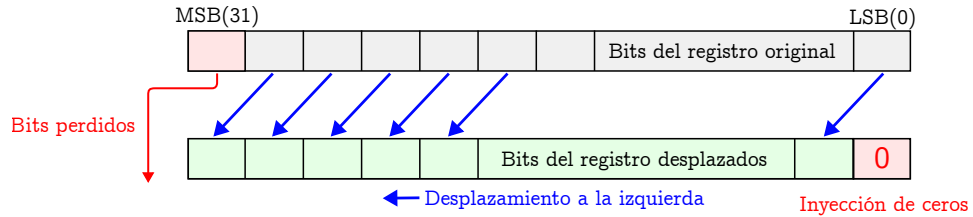


Figura 7.1: Desplazamiento lógico hacia la izquierda (SLL).

Los desplazamientos lógicos a la derecha se utilizan normalmente para operar con números enteros “sin signo” (*unsigned integer division*). Desplazar un número sin signo hacia la derecha n posiciones equivale a una división entera:

$$Y = \lfloor \frac{X}{2^n} \rfloor \quad (7.2)$$

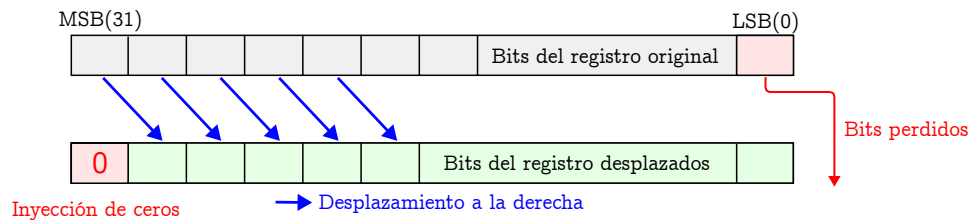


Figura 7.2: Desplazamiento lógico hacia la derecha (SRL).

Las figuras 7.1 y 7.2 muestra el funcionamiento de los desplazamientos lógicos.

Sintaxis y ejemplos

```
sll $t0, $t1, 4
srl $t2, $t0, 8
```

Si inicialmente $t1 = 0x12345678$ la ejecución produce $t0 = 0x23456780$ y posteriormente $t2 = 0x00234567$

7.4.2. Desplazamiento aritmético

La instrucción **SRA** realiza un desplazamiento hacia la derecha preservando el signo del operando. Las posiciones vacantes se completan replicando el bit más significativo del registro.

$$R[rd] = R[rt] \ggg param$$

La Figura 7.3 ilustra el comportamiento del desplazamiento aritmético.

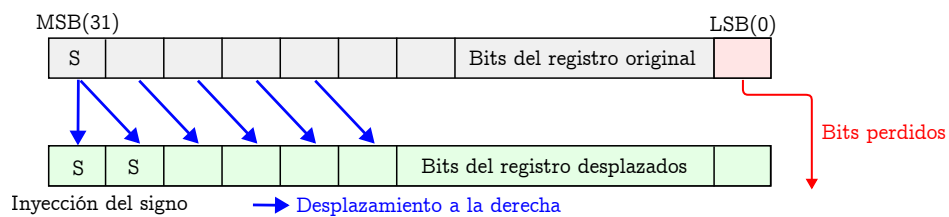


Figura 7.3: Desplazamiento aritmético hacia la derecha (SRA).

Sintaxis y ejemplos

```
sra $t0, $t1, 3
```

Si inicialmente $t1 = 0xFFFFFFFF80$ la ejecución produce $t0 = 0xFFFFFFFFF0$.

7.4.3. Rotación

La instrucción RLC realiza una rotación circular hacia la izquierda. A diferencia de un desplazamiento lógico, los bits que abandonan el extremo más significativo del registro vuelven a introducirse por el extremo menos significativo, por lo que no se pierde información.

$$R[rd] = R[rt] \llcorner param$$

La Figura 7.4 muestra el funcionamiento de esta operación.

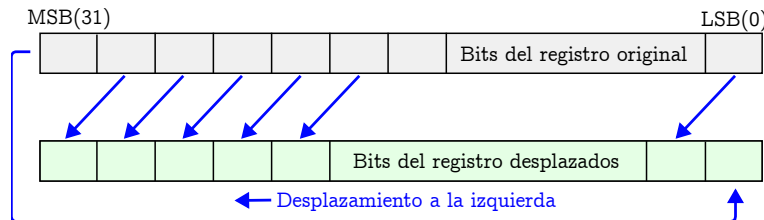


Figura 7.4: Rotación circular hacia la izquierda (RLC).

Sintaxis y ejemplos

```
rlc $t0, $t1, 8
```

Si inicialmente $t1 = 0x12345678$ la ejecución produce $t0 = 0x34567812$.

7.4.4. Variantes controladas por registro

Las instrucciones SLLR, SRLR, SRAR y RLCR realizan las mismas operaciones que sus variantes con desplazamiento inmediato. La diferencia consiste en que la cantidad de posiciones a desplazar se obtiene de los cinco bits menos significativos del registro **rs**.

Instrucción	Operación
SLLR	$R[rd] = R[rt] \ll R[rs]_{[4:0]}$
SRLR	$R[rd] = R[rt] \gg R[rs]_{[4:0]}$
SRAR	$R[rd] = R[rt] \ggg R[rs]_{[4:0]}$
RLCR	$R[rd] = R[rt] \llcorner R[rs]_{[4:0]}$

Sintaxis y ejemplos

```
sllr $t0, $a0, $t1
rlcr $t2, $a0, $t1
```


Si inicialmente $a0 = 8$ y $t1 = 0x02000001$ la primera instrucción desplaza el contenido de $t1$ ocho posiciones hacia la izquierda, mientras que la segunda realiza una rotación circular a la izquierda de la misma magnitud quedando $t2 = 0x00000012$.

NOTA

La arquitectura implementa únicamente la rotación circular hacia la izquierda mediante las instrucciones RLC y RLCR. Esta decisión simplifica el conjunto de instrucciones sin reducir su capacidad funcional, ya que una rotación hacia la derecha puede obtenerse mediante una rotación hacia la izquierda de $32 - n$ posiciones. De esta forma se evita incorporar instrucciones redundantes manteniendo la simetría del conjunto de operaciones de desplazamiento.

7.5. Instrucciones de comparación

Las instrucciones de comparación permiten evaluar la relación de orden entre dos operandos y almacenar el resultado directamente en un registro de propósito general. A diferencia de las instrucciones aritméticas, estas instrucciones no producen un resultado numérico derivado de los operandos, sino un valor booleano representado mediante un entero.

Esta familia está formada por cuatro instrucciones. SLT y SLTU comparan el contenido de dos registros, mientras que SLTI y SLTIU comparan un registro con un operando inmediato.

El resultado de la comparación siempre es un valor entero. Si la condición evaluada es verdadera, el registro destino recibe el valor 1; en caso contrario recibe el valor 0.

Las instrucciones de esta familia no modifican las banderas de estado.

7.5.1. Comparación entre registros

Las instrucciones SLT y SLTU comparan el contenido de dos registros de propósito general y almacenan el resultado de la comparación en un tercer registro.

La diferencia entre ambas instrucciones radica en la interpretación de los operandos. SLT realiza una comparación con signo utilizando la representación en complemento a dos, mientras que SLTU interpreta ambos operandos como enteros sin signo mediante el operador $U()$ definido previamente.

Instrucción	Operación
SLT	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$
SLTU	$R[rd] = (U(R[rs]) < U(R[rt])) ? 1 : 0$

Sintaxis y ejemplos

```
slt  $t0, $a0, $a1
sltu $t1, $a2, $a3
```

Si inicialmente $a0 = -8$ y $a1 = 5$ la primera comparación produce $t0 = 1$ ya que $-8 < 5$. En cambio, SLTU interpreta ambos operandos como enteros sin signo, por lo que resulta adecuada para comparar direcciones de memoria, tamaños, índices y otros valores que no poseen interpretación con signo.

7.5.2. Comparación con operandos inmediatos

Las instrucciones **SLTI** y **SLTIU** realizan la comparación entre un registro y un operando inmediato codificado dentro de la instrucción. En **SLTI** el operando inmediato se interpreta como un entero con signo y se extiende mediante el operador $E_{17}(\text{imm}, S)$, mientras que **SLTIU** realiza una extensión con ceros utilizando $E_{17}(\text{imm}, 0)$ para efectuar una comparación sin signo.

Instrucción	Operación
SLTI	$R[rt] = (R[rs] < E_{17}(\text{imm}, S)) ? 1 : 0$
SLTIU	$R[rt] = (U(R[rs]) < U(E_{17}(\text{imm}, 0))) ? 1 : 0$

Sintaxis y ejemplos

```
slti $t0, $s0, 100
sltiu $t1, $s1, 1024
```

Si inicialmente $s0 = 75$ la ejecución produce $t0 = 1$ ya que $75 < 100$.

La instrucción **SLTIU** resulta especialmente útil para verificar si un valor sin signo pertenece a un determinado rango o es inferior a un límite constante.

7.5.3. Resultados booleanos

El resultado producido por las instrucciones de comparación constituye un valor booleano almacenado en un registro de propósito general. A diferencia de las arquitecturas que representan el resultado de una comparación exclusivamente mediante banderas de estado, RTM32 permite utilizar directamente dicho valor como un operando más dentro del programa.

El resultado de una comparación puede almacenarse en memoria, transferirse como argumento de una función, combinarse mediante instrucciones lógicas o emplearse posteriormente para controlar el flujo de ejecución.

Por ejemplo:

```
slt $t0, $a0, $a1
beq $t0, $zero, mayor_igual
```

En este caso, la instrucción **SLT** genera el valor booleano correspondiente a la comparación, mientras que la instrucción de salto, que se verá más adelante, utiliza dicho resultado para decidir el flujo de ejecución.

7.6. Instrucciones de acceso a memoria

Las instrucciones de acceso a memoria permiten transferir datos entre la memoria principal y los registros de propósito general. En RTM32 todas las operaciones de carga y almacenamiento se realizan exclusivamente mediante este conjunto de instrucciones; las operaciones aritméticas y lógicas nunca operan directamente sobre memoria.

La arquitectura implementa un modelo *Load/Store*, donde las instrucciones de carga copian datos desde memoria hacia un registro, mientras que las instrucciones de almacenamiento realizan la operación inversa.

RTM32 soporta accesos de palabra (32 bits), media palabra (16 bits) y byte (8 bits), incluyendo variantes con y sin extensión de signo. Además, proporciona dos mecanismos para calcular la dirección efectiva:

- Direccionamiento indexado mediante un registro base y un desplazamiento inmediato ($EA()$).
- Direccionamiento indexado mediante dos registros ($EAX()$).

Las instrucciones de carga actualizan las banderas de estado Z y N de acuerdo con el valor cargado en el registro destino. Las instrucciones de almacenamiento no modifican las banderas del procesador.

La familia está formada por las instrucciones LW, LWX, SW, SWX, LH, LHU, LHX, LHUX, SH, SHX, LB, LBU, LBX, LBUX, SB y SBX.

7.6.1. Carga y almacenamiento de palabras

Las instrucciones de palabra transfieren un valor completo de 32 bits entre la memoria y un registro de propósito general.

Instrucción	Operación
LW	$R[rt] = M[EA(rs, imm)]$
LWX	$R[rt] = M[EAX(rs, rd)]$
SW	$M[EA(rs, imm)] = R[rt]$
SWX	$M[EAX(rs, rd)] = R[rt]$

Las instrucciones LW y SW utilizan direccionamiento indexado con desplazamiento inmediato, mientras que LWX y SWX calculan la dirección efectiva utilizando dos registros.

Sintaxis y ejemplos

```
lw    $t0, 8($sp)
lw    $t1, $s0, $a0
sw    $t2, -4($fp)
```

7.6.2. Carga y almacenamiento de medias palabras

Las instrucciones de media palabra permiten acceder a datos de 16 bits.

Las variantes de carga se diferencian únicamente por el mecanismo utilizado para extender el dato leído hasta el tamaño nativo del procesador.

Instrucción	Operación
LH	$R[rt] = E_{16}(M[EA(rs, imm)], S)$
LHU	$R[rt] = E_{16}(M[EA(rs, imm)], 0)$
LHX	$R[rt] = E_{16}(M[EAX(rs, rd)], S)$
LHUX	$R[rt] = E_{16}(M[EAX(rs, rd)], 0)$
SH	$M[EA(rs, imm)] = R[rt]_{[15:0]}$
SHX	$M[EAX(rs, rd)] = R[rt]_{[15:0]}$

Las instrucciones LH y LHX realizan extensión con signo, mientras que LHU y LHUX realizan extensión con ceros. Las instrucciones SH y SHX almacenan únicamente los dieciséis bits menos significativos del registro fuente.

Sintaxis y ejemplos

```
lh    $t0, 2($sp)
lhu   $t1, 4($sp)
lhx   $t2, $s0, $a0
lhux  $t3, $s1, $a1
sh    $t4, 6($sp)
```

7.6.3. Carga y almacenamiento de bytes

Las instrucciones de byte permiten acceder individualmente a cada uno de los bytes almacenados en memoria.

Al igual que en las instrucciones de media palabra, las variantes de carga se diferencian únicamente por el tipo de extensión aplicado al dato leído.

Instrucción	Operación
LB	$R[rt] = E_8(M[EA(rs, imm)], S)$
LBU	$R[rt] = E_8(M[EA(rs, imm)], 0)$
LBX	$R[rt] = E_8(M[EAX(rs, rd)], S)$
LBUX	$R[rt] = E_8(M[EAX(rs, rd)], 0)$
SB	$M[EA(rs, imm)] = R[rt]_{[7:0]}$
SBX	$M[EAX(rs, rd)] = R[rt]_{[7:0]}$

Las instrucciones LB y LBX preservan el signo del byte leído, mientras que LBU y LBUX completan los bits superiores con ceros. Las instrucciones SB y SBX almacenan únicamente el byte menos significativo del registro fuente.

Sintaxis y ejemplos

```
lb    $a0, 0($s0)
lbu   $a1, 1($s0)
lbx   $v0, $s1, $t0
lbux  $v1, $s2, $t1
sb    $t2, 3($sp)
```

ATENCIÓN

Las instrucciones que operan sobre palabras requieren que la dirección efectiva se encuentre alineada a cuatro bytes, mientras que las instrucciones de media palabra requieren alineación a dos bytes. Las instrucciones de acceso a bytes no poseen restricciones de alineación. Cuando una instrucción utiliza una dirección que no satisface los requisitos de alineación establecidos por la arquitectura, la CPU genera la excepción correspondiente.

7.7. Instrucciones de transferencia del control

Las instrucciones de transferencia del control modifican el valor del contador de programa (PC), permitiendo alterar la secuencia normal de ejecución. Esta familia implementa tanto saltos condicionales como incondicionales, llamadas a procedimientos y transferencias indirectas de control.

RTM32 utiliza dos mecanismos para calcular la dirección de destino:

- Direccionamiento relativo al contador de programa mediante la función $RA()$.
- Direccionamiento relativo a un registro base mediante la función $AA()$.

Las instrucciones de salto no modifican las banderas de estado del procesador.

La familia está formada por las instrucciones BEQ, BNE, BLT, BGE, BLTU, BGEU, J, JAL, JALX, JR, JALR y JALRX.

7.7.1. Saltos condicionales

Las instrucciones de salto condicional comparan dos registros y modifican el contador de programa únicamente cuando la condición evaluada resulta verdadera. El destino del salto se calcula mediante la función $RA()$, utilizando un desplazamiento relativo al contador de programa.

Instrucción	Condición
BEQ	$R[rs] = R[rt]$
BNE	$R[rs] \neq R[rt]$
BLT	$R[rs] < R[rt]$
BGE	$R[rs] \geq R[rt]$
BLTU	$U(R[rs]) < U(R[rt])$
BGEU	$U(R[rs]) \geq U(R[rt])$

Las comparaciones BLT y BGE interpretan ambos operandos como enteros con signo, mientras que BLTU y BGEU realizan la comparación sobre valores sin signo.

Simetría de las comparaciones

RTM32 implementa únicamente las comparaciones “menor que” y “mayor o igual que”, tanto para enteros con signo como sin signo.

Esta decisión reduce el número de instrucciones necesarias sin disminuir la capacidad expresiva de la arquitectura. Las relaciones restantes pueden obtenerse invirtiendo el orden de los operandos o utilizando la condición complementaria.

Por ejemplo,

$$A > B$$

es equivalente a

$$B < A$$

mientras que

$$A \leq B$$

es equivalente a

$$B \geq A.$$

De esta forma la arquitectura mantiene un conjunto de comparaciones mínimo y simétrico, simplificando el decodificador y reduciendo el espacio de codificación requerido por la ISA.

Sintaxis y ejemplos

```
beq  $t0,$t1,label
bne  $a0,$zero,error
blt  $s0,$s1,loop
bge  $sp,$fp,exit
bltu $t0,$t1,unsigned_cmp
bgeu $t2,$t3,unsigned_end
```

7.7.2. Saltos relativos incondicionales

Las instrucciones de salto relativo modifican el contador de programa sin realizar comparaciones previas. El destino se calcula mediante la función `RA()`.

Instrucción	Operación
J	$R[0] = PC + 4; PC = RA(limm)$
JAL	$R[1] = PC + 4; PC = RA(limm)$
JALX	$R[lrx] = PC + 4; PC = RA(llimm)$

La instrucción J realiza un salto incondicional sin conservar la dirección de retorno.

Las instrucciones JAL y JALX almacenan previamente la dirección de la siguiente instrucción en un registro de enlace antes de efectuar el salto. Mientras JAL utiliza el registro de enlace predeterminado, JALX permite seleccionar cualquiera de los registros de enlace definidos por la arquitectura.

Sintaxis y ejemplos

```
j    loop
jal  printf
jalx $lr2,handler
```

7.7.3. Saltos indirectos incondicionales

Las instrucciones de salto indirecto calculan el destino a partir del contenido de un registro base y un desplazamiento inmediato mediante la función `AA()`.

Instrucción	Operación
JR	$R[0] = PC + 4; PC = AA(rs, imm)$
JALR	$R[1] = PC + 4; PC = AA(rs, imm)$
JALRX	$R[lrx] = PC + 4; PC = AA(rs, llim)$

Este mecanismo resulta apropiado para implementar llamadas indirectas, despacho mediante tablas de funciones, punteros a funciones y otras formas de transferencia dinámica del control.

Al igual que en los saltos relativos, las variantes con enlace almacenan la dirección de retorno antes de actualizar el contador de programa.

Sintaxis y ejemplos

```
jr    $t0, 0
jalr  $s0, 8
jalrx $lr3, $gp, -4
```

NOTA

Todas las instrucciones de transferencia del control utilizan direcciones relativas, ya sea respecto del contador de programa ($RA()$) o de un registro base ($AA()$). La arquitectura no incorpora instrucciones que codifiquen direcciones absolutas dentro de la propia instrucción.

7.8. Instrucciones de sistema

Las instrucciones de sistema permiten interactuar con recursos internos del procesador que no forman parte del conjunto de registros de propósito general. Estas instrucciones se utilizan para acceder al estado de la CPU, modificar registros especiales y realizar la transición entre los modos de ejecución de usuario y supervisor.

A diferencia del resto de las instrucciones de la arquitectura, las instrucciones de sistema operan sobre recursos privilegiados cuyo acceso puede estar restringido por el modo de ejecución actual del procesador.

La familia está formada por las instrucciones CFS, CTS, TRAP y RFT.

7.8.1. Acceso a registros especiales

Las instrucciones CFS (*Copy From Special*) y CTS (*Copy To Special*) permiten transferir información entre los registros de propósito general y los registros especiales del procesador.

Ambas instrucciones utilizan el formato R. El campo **param** identifica el registro especial involucrado en la operación.

Instrucción	Operación
CFS	$R[rs] = S[param]$
CTS	$S[param] = R[rs]$

Estas instrucciones constituyen el mecanismo general utilizado por la arquitectura para leer y modificar el estado interno del procesador. La descripción de cada registro especial y de sus privilegios de acceso se presenta en el capítulo correspondiente.

Sintaxis y ejemplos

```
cfs $t0,PSW
cts $t1,EPC
```

7.8.2. Generación de excepciones

La instrucción **TRAP** provoca una excepción software que transfiere el control al manejador de excepciones correspondiente (normalmente el sistema operativo).

Antes de modificar el flujo de ejecución, la CPU almacena la dirección de la siguiente instrucción en el registro especial **EPC** y guarda el modo de ejecución actual en el registro **ESR**. A continuación, conmuta al modo supervisor y obtiene la dirección del manejador correspondiente mediante la función **TV()** y transfiere el control a dicha rutina.

Instrucción	Operación
TRAP	$EPC = PC + 4;$ $ESR = PSW;$ $PSW.MODE = Supervisor;$ $PSW.IE = 0;$ $PSW.TE = 0;$ $PC = M[TV(param)]$

El operando inmediato selecciona una entrada de la tabla de vectores, por lo que una instrucción **TRAP** puede invocar cualquier manejador definido por la arquitectura. De esta forma, las excepciones software y las interrupciones hardware comparten una única tabla de vectores, diferenciándose únicamente por la entrada utilizada.

La ejecución de **TRAP** deshabilita automáticamente la aceptación de interrupciones y de nuevas excepciones software. El manejador puede rehabilitarlas modificando el registro **PSW** mediante las instrucciones **CFS** y **CTS**.

Durante esta operación el procesador abandona el modo usuario y continúa la ejecución en modo supervisor, permitiendo que el núcleo del sistema operativo o el correspondiente manejador atienda la excepción o el servicio solicitado.

La instrucción puede ejecutarse siempre desde modo usuario. Cuando el procesador ya se encuentra ejecutando en modo supervisor, la ejecución de **TRAP** requiere que el bit **TE** (*Supervisor Trap Enable*) del registro **PSW** se encuentre habilitado. Esta restricción evita excepciones software recursivas no deseadas. En ambos casos la CPU preserva el modo de ejecución previo para que pueda ser restaurado posteriormente por la instrucción **RFT**.

Sintaxis y ejemplos

```
trap 5
```


7.8.3. Retorno de excepción

La instrucción **RFT** (*Return From Trap*) finaliza la ejecución del manejador de excepción y restaura el contexto de ejecución previamente almacenado por la CPU.

Instrucción	Operación
RFT	$PSW = ESR;$ $PC = EPC$

Durante su ejecución, la CPU restaura el modo de ejecución almacenado en el registro **ESR** y recupera el valor del contador de programa desde **EPC**. Como consecuencia, la ejecución continúa exactamente en la instrucción siguiente a la que originó la excepción.

La instrucción **RFT** constituye el único mecanismo arquitectónico para abandonar el modo supervisor y retornar al contexto previamente interrumpido.

Sintaxis y ejemplos

```
rft
```

NOTA

La arquitectura utiliza una tabla de vectores unificada para excepciones e interrupciones. Las excepciones software generadas mediante **TRAP** y los eventos hardware utilizan el mismo mecanismo de despacho, diferenciándose únicamente por la entrada de la tabla de vectores seleccionada.

Parte IV

Operación del Sistema

8. Excepciones

PRONTO ...

Parte V

Dispositivos y Entrada/Salida

9. Entrada/Salida

PRONTO ...

Parte VI

Interface de Software

9.1. Compilador RTM32AS

PRONTO ...

Parte VII

Verificación y Debugging

10. Interfaz del Depurador (Modo Debug)

10.1. Activación del Entorno Interactivo

El emulador arranca por defecto en modo directo de alta velocidad. Para iniciar la consola interactiva de depuración comandada por terminal, utilice los modificadores de lanzamiento:

```
./rtm32_emu -d  
# O alternativamente:  
./rtm32_emu --debug
```

10.2. Diccionario de Comandos Aceptados

La consola de comandos responde al prompt RTM32> mediante una interfaz síncrona. Los comandos implementados son los siguientes:

10.2.1. Comando G (Ejecución Continua o Limitada)

Sintaxis: G x [n]

Establece el PC en la dirección hexadecimal x e inicia la ejecución. Si se especifica el parámetro opcional n, la CPU procesará exactamente n instrucciones antes de pausarse y devolver el control al depurador. Si se omite n, corre de forma indefinida hasta el final o una instrucción de detención.

10.2.2. Comando D R (Vuelco de Registros)

Sintaxis: D R[x]

Si se ejecuta de forma aislada como D R, vuelca por pantalla el contenido actual de los 32 registros y el contador de programa (PC). Si se indica un índice específico (ej. D R5), imprime el valor único del registro seleccionado.

10.2.3. Comando D M (Vuelco de Memoria)

Sintaxis: D Mx [n]

Muestra el contenido hexadecimal de la palabra en la dirección x. Si se proporciona n, realiza un volcado en ráfaga consecutiva de n direcciones de memoria.

10.2.4. Comando W (Escritura en Memoria)

Sintaxis: W x v

Escribe de forma forzada el valor hexadecimal v en la dirección de memoria especificada por x.

10.2.5. Comandos S y L (Gestión de Estados e Instantáneas)

Sintaxis: S filename / L filename

Permiten serializar y deserializar de forma directa el bloque binario completo de la estructura de datos CPU (incluyendo registros, estado de periféricos seriales y el bloque completo

de memoria RAM) hacia o desde el archivo local indicado por **filename**, garantizando la persistencia completa de las sesiones de depuración.

A. Notation

Notación	Descripción
$R[x]$	Registro de propósito general con índice x .
$S[x]$	Registro especial con índice x .
$M[x]$	Contenido de la posición de memoria ubicada en la dirección x .
PC	Contador de programa.
EPC	Contador de programa almacenado durante una excepción.
$x_{[m:n]}$	Selección del campo de bits comprendido entre las posiciones m y n .
$x \parallel y$	Concatenación de las secuencias binarias x e y .
$E_x(y, z)$	Extensión de un operando de x bits mediante el mecanismo $z \in \{S, 0, 1\}$.
$C_x(y, z)$	Concatenación de los x bits de y con los x bits menos significativos de z .
$U(x)$	Interpretación de x como entero sin signo.
$EA(rs, imm)$	Dirección efectiva para accesos a memoria con desplazamiento inmediato.
$EAX(rs, rd)$	Dirección efectiva calculada a partir de dos registros.
$RA_x(imm)$	Dirección relativa al contador de programa.
$AA_x(rs, imm)$	Dirección de salto relativa a un registro base.
$TV(param)$	Desplazamiento correspondiente a una entrada de la tabla de vectores.

Tabla A.1: Resumen de la notación utilizada en la especificación de RTM32

B. Instruction Set Reference

Opcode	T	Mnemo	Operands	Operation
00000	R			see table B.2
00001/00	J	J	vlimm	$R[0] = PC + 4; PC = RA_{27}(\text{limm})$ ⁽¹⁾
00001/01	J	JAL	vlimm	$R[1] = PC + 4; PC = RA_{27}(\text{limm})$
00001/1ix	J	JALX	limm	$R[1ix] = PC + 4; PC = RA_{26}(\text{limm})$
00010/00	I	JR	rs llimm	$R[0] = PC + 4; PC = AA_{22}(\text{rs}, \text{limm})$ ⁽²⁾
00010/01	I	JALR	rs llimm	$R[1] = PC + 4; PC = AA_{22}(\text{rs}, \text{limm})$
00010/1ix	I	JALRX	rs slimm	$R[1ix] = PC + 4; PC = AA_{21}(\text{rs}, \text{llimm})$
00011	I	ADDI	rs rt simm	$R[\text{rt}] = R[\text{rs}] + E_{17}(\text{simm}, S)$ ⁽³⁾
00100/0	I	ANDI	rs rt simm	$R[\text{rt}] = R[\text{rs}] \& E_{16}(\text{simm}, 0)$
00100/1	I	LCI	rs rt simm	$R[\text{rt}] = C_{16}(\text{simm}, R[\text{rs}])$ ⁽⁴⁾
00101/0	I	ANI	rs rt simm	$R[\text{rt}] = R[\text{rs}] \& E_{16}(\text{simm}, 1)$
00101/1	I	ANH	rs rt simm	$R[\text{rt}] = R[\text{rs}] \& C_{16}(\text{simm}, R[\text{rs}])$
00110/0	I	ORI	rs rt simm	$R[\text{rt}] = R[\text{rs}] E_{16}(\text{simm}, 0)$
00110/1	I	ORH	rs rt simm	$R[\text{rt}] = R[\text{rs}] C_{16}(\text{simm}, \$0)$
00111/0	I	XORI	rs rt simm	$R[\text{rt}] = R[\text{rs}] \oplus E_{16}(\text{simm}, 0)$
00111/1	I	XORH	rs rt simm	$R[\text{rt}] = R[\text{rs}] \oplus C_{16}(\text{simm}, \$0)$
01000	I	LW	rs rt imm	$R[\text{rt}] = M[\text{EA}(\text{rs}, \text{imm})]$ ⁽⁵⁾
01001	I	SW	rs rt imm	$M[\text{EA}(\text{rs}, \text{imm})] = R[\text{rt}]$
01010	I	SH	rs rt imm	$M[\text{EA}(\text{rs}, \text{imm})] = R[\text{rt}]_{[15:0]}$
01011	I	SB	rs rt imm	$M[\text{EA}(\text{rs}, \text{imm})] = R[\text{rt}]_{[7:0]}$
01100	I	LH	rs rt imm	$R[\text{rt}] = E_{16}(M[\text{EA}(\text{rs}, \text{imm})], S)$
01101	I	LHU	rs rt imm	$R[\text{rt}] = E_{16}(M[\text{EA}(\text{rs}, \text{imm})], 0)$
01110	I	LB	rs rt imm	$R[\text{rt}] = E_8(M[\text{EA}(\text{rs}, \text{imm})], S)$
01111	I	LBU	rs rt imm	$R[\text{rt}] = E_8(M[\text{EA}(\text{rs}, \text{imm})], 0)$
10000	I	BEQ	rs rt imm	if $(R[\text{rs}] == R[\text{rt}])$ $PC = RA_{19}(\text{imm})$
10001	I	BNE	rs rt imm	if $(R[\text{rs}] != R[\text{rt}])$ $PC = RA_{19}(\text{imm})$
10010	I	BLT	rs rt imm	if $(R[\text{rs}] < R[\text{rt}])$ $PC = RA_{19}(\text{imm})$ ⁽⁶⁾
10011	I	BGE	rs rt imm	if $(R[\text{rs}] >= R[\text{rt}])$ $PC = RA_{19}(\text{imm})$
10100	I	BLTU	rs rt imm	if $(U(R[\text{rs}]) < U(R[\text{rt}]))$ $PC = RA_{19}(\text{imm})$
10101	I	BGEU	rs rt imm	if $(U(R[\text{rs}]) >= U(R[\text{rt}]))$ $PC = RA_{19}(\text{imm})$
10110	I	SLTI	rs rt imm	$R[\text{rt}] = (R[\text{rs}] < E_{17}(\text{imm}, S)) ? 1 : 0$
10111	I	SLTIU	rs rt imm	$R[\text{rt}] = (U(R[\text{rs}]) < U(E_{17}(\text{imm}, 0))) ? 1 : 0$

Tabla B.1: RTM32 Instruction Set sorted by opcode

- (1) $RA_x(y) = PC + 4 + E_x(4y, S)$ calcula la dirección en palabras relativa al PC actual.
- (2) $AA_x(\text{rs}, \text{imm}) = R[\text{rs}] + E_x(4\text{imm}, S)$ calcula la dirección absoluta en palabras relativa a $R[\text{rs}]$.
- (3) $E_x(y, z)$ extiende los x bits de y con z $\in S, 0, 1$ (signo, cero o uno).
- (4) $C_x(y, z)$ concatena los x bits de y como parte más significativa con los x bits menos significativos de z.
- (5) $EA(\text{rs}, \text{imm}) = R[\text{rs}] + E_{17}(\text{imm}, S)$ calcula la dirección efectiva.
- (6) $U(x) = (\text{unsigned})x$ valor no signado. Salvo que se indique explícitamente lo contrario, todos los operadores relacionales ($<$, $<=$, $>$, $>=$) realizan comparaciones con signo. Para efectuar comparaciones sin signo debe utilizarse la notación $U(x)$ sobre los operandos correspondientes.

Func	Mnemo	Operands	Operation
000000	SLL	rt rd param	$R[rd] = R[rt] \ll \text{param}$
000001	RLC	rt rd param	$R[rd] = R[rt] \lll \text{param}$
000010	SRL	rt rd param	$R[rd] = R[rt] \gg \text{param}$
000011	SRA	rt rd param	$R[rd] = R[rt] \ggg \text{param}$
000100	SLLR	rs rt rd	$R[rd] = R[rt] \ll R[rs]_{[4:0]}$
000101	RLCR	rs rt rd	$R[rd] = R[rt] \lll R[rs]_{[4:0]}$
000110	SRLR	rs rt rd	$R[rd] = R[rt] \gg R[rs]_{[4:0]}$
000111	SRAR	rs rt rd	$R[rd] = R[rt] \ggg R[rs]_{[4:0]}$
001000	AND	rs rt rd	$R[rd] = R[rs] \& R[rt]$
001001	OR	rs rt rd	$R[rd] = R[rs] \mid R[rt]$
001010	XOR	rs rt rd	$R[rd] = R[rs] \oplus R[rt]$
001011	NOR	rs rt rd	$R[rd] = \neg (R[rs] \mid R[rt])$
001100	ADD	rs rt rd	$R[rd] = R[rs] + R[rt]$
001101	SUB	rs rt rd	$R[rd] = R[rs] - R[rt]$
001110	SLT	rs rt rd	$R[rd] = (R[rs] < R[rt]) ? 1 : 0$
001111	SLTU	rs rt rd	$R[rd] = (U(R[rs]) < U(R[rt])) ? 1 : 0$
010000	LHX	rs rt rd	$R[rt] = E_{16}(M[EAX(rs,rd)], S)^{(7)}$
010001	LHUX	rs rt rd	$R[rt] = E_{16}(M[EAX(rs,rd)], 0)$
010010	LBX	rs rt rd	$R[rt] = E_8(M[EAX(rs,rd)], S)$
010011	LBUX	rs rt rd	$R[rt] = E_8(M[EAX(rs,rd)], 0)$
010100	LWX	rs rt rd	$R[rt] = M[EAX(rs,rd)]$
010101	SWX	rs rt rd	$M[EAX(rs,rd)] = R[rt]$
010110	SHX	rs rt rd	$M[EAX(rs,rd)] = R[rt]_{[15:0]}$
010111	SBX	rs rt rd	$M[EAX(rs,rd)] = R[rt]_{[7:0]}$
011000	MUL	rs rt rd	$R[rd] = \{R[rs] \times R[rt]\}_{[31:0]}$
011001	MULH	rs rt rd	$R[rd] = \{R[rs] \times R[rt]\}_{[63:32]}$
011010	MULHU	rs rt rd	$R[rd] = \{U(R[rs]) \times U(R[rt])\}_{[63:32]}$
011011	MULHSU	rs rt rd	$R[rd] = \{R[rs] \times U(R[rt])\}_{[63:32]}$
011100	DIV	rs rt rd	$R[rd] = R[rs] / R[rt]$
011101	DIVU	rs rt rd	$R[rd] = U(R[rs]) / U(R[rt])$
011110	REST	rs rt rd	$R[rd] = R[rs] \% R[rt]$
011111	RESTU	rs rt rd	$R[rd] = U(R[rs]) \% U(R[rt])$
100000	CFS	rs param	$R[rs] = S[\text{param}]$
100001	CTS	rs param	$S[\text{param}] = R[rs]$
100010	TRAP	param	$EPC = PC + 4; PC = M[TV(\text{param})]^{(8)}$
100011	RFT		$PC = EPC$

Tabla B.2: R-type instructions sorted by func

(7) $EAX(rs,rd) = R[rs] + R[rd]$ calcula la dirección efectiva extendida por otro registro **EAX**

(8) $TV(\text{param}) = \text{param} \ll 2$